



**University of
Nottingham**

UK | CHINA | MALAYSIA

Robustness in Neural Networks

Submitted April 26, 2021, in partial fulfilment of the conditions for the award of the degree of
BSc Hons Computer Science with Artificial Intelligence

Can Zhou

20031818

Supervised by Dr. Amin Farjudian

I hereby declare that this dissertation is all my own work, except as indicated in the text:

Signature _____

Date 04 / 26 / 2021

I hereby declare that I have all necessary rights and consents to publicly distribute this dissertation via the University of Nottingham's e-dissertation archive.

**School of Computer Science
University of Nottingham, Ningbo China**

Abstract

Certifying robustness is a critical issue in modern adversarial machine learning due to security implications. The (local) Lipschitz constant of the neural network has been determined to have a substantial impact on robustness estimation and this estimating mechanism is attack-agnostic. However, estimating the exact Lipschitz constant is a known NP-hard problem. Therefore, any method that is scalable to large networks—including the cutting-edge methods of the literature—must be unverified, and many are shown to be easily prone to errors. Furthermore, some important activation functions in neural networks, such as *ReLU*, are non-differentiable. As a result, one cannot apply classical differentiation and chain rule to correctly compute the Lipschitz constant of non-differentiable networks through back-propagation.

In this project, we propose a novel interval framework for validated estimation of Lipschitz constant of neural networks. With the aid of interval computation and Clarke’s generalized gradient, our framework is able to analyse non-differentiable networks with guarantees correctness.

We choose multi-dimensional bisection as the main methodology in our framework and present an implementation of the framework using multiple-precision interval arithmetic. The results of our experiments demonstrate how our framework may be used as a verifier for other robustness analysers. In particular, we demonstrate how some of the well-known statistical methods of the literature underestimate the true Lipschitz constant in some critical regions of the input space, leading to false negatives, which must be avoided in safety critical systems. Moreover, by using interval arithmetic, our framework can locate the regions of least robustness which can be used for further robustness enhancement.

As the original goals of the project proposal have been achieved earlier than planned, we investigate another possible approach—i.e., interval Newton method—to make our framework more efficient. The literature on interval Newton method is developed for continuously differentiable functions, whereas for robustness analysis, one must handle Lipschitz (and at time, non-differentiable) functions. To that end, we adopt Clarke gradient again and relax the prerequisite of this method from continuously differentiable to Lipschitz continuous.

Acknowledgements

First and foremost, I would like to express my deep and sincere gratitude to my supervisor: Dr. Amin Farjudian who constantly supports me with his patience, encouragement, enthusiasm and immense knowledge. Anatole France said: "If the path be beautiful, let us not ask where it leads". Thanks to my supervisor, I not only explored an enlightening and enjoyable journey in my final year, but also stayed on track with the correct research direction throughout the project. I could not have wished for a better supervisor and it has been a privilege to work on this project with him.

In addition, I would like to extend my heartfelt thanks to all staff in the school of computer science, for helping me learn relevant background knowledge; my parents, for offering me unwavering trust and always supporting me mentally as well as financially; my friends, for bringing me so much joy through the years.

Finally, many thanks go to all those who helped me in any way in this project and a number of mostly open source projects and tools, such as GNU/Linux, GCC, Boost, Python, Pytorch, Emacs, L^AT_EX—we should never take them for granted.

Contents

1	Introduction	1
1.1	Background	1
1.2	Motivation	1
1.3	Aims and Objectives	2
1.4	Contributions	2
1.5	Dissertation Outline	3
2	Preliminaries	4
2.1	Lipschitz Continuity	5
2.2	Clarke Gradient	5
2.3	Interval Arithmetic	6
2.3.1	Interval Arithmetic Operations	7
2.3.2	Finite Representations	7
2.3.3	Dependency	7
2.4	Neural Network	8
3	Related Work	9
3.1	Direct Robustness Measurement	9
3.1.1	Theoretical Robustness Guarantees for Neural Networks	9
3.1.2	CLEVER	9
3.2	Indirect Robustness Measurement	10
3.2.1	Fast-Lip	10
3.2.2	LipSDP	11
3.2.3	SeqLip	11
3.3	Discussion	12
4	Methodology	13
4.1	Interval Methodologies	13
4.1.1	Multi-dimensional Bisection	13
4.1.2	Interval Newton Methods	13

4.1.3	Discussion	14
4.2	Symbolic Computation	15
4.3	Project Methodologies	16
4.3.1	Development Methodology	16
4.3.2	Testing Methodology	16
4.3.3	Version Control Methodology	16
5	Design	17
5.1	Bisection-based Maximization Algorithm	17
5.2	Interval Newton-based Algorithm	17
5.3	Prototype of Automatic Measuring Framework	18
6	Implementation	21
6.1	Environment, Languages, and Libraries	21
6.2	Implementation of Automatic Measuring System	21
6.3	List of Functions Used in the Project	22
7	Evaluation	23
7.1	Experiment Setup	23
7.2	Differentiable Activation Function	24
7.3	Non-Differentiable Activation Function	24
7.4	Critical Regions Near Boundaries	24
7.5	Finding the Critical Regions	25
7.6	Interval Newton Method	27
8	Summary and Reflections	29
8.1	Contributions	29
8.2	Future Directions	30
8.3	Project Management	30
8.4	Reflections	32
	References	32

List of Figures

1.1	One Pixel Attack: the output of NNs can be easily altered by changing one pixel in the inputs	1
2.1	Dependency problem: The following network computes the constant function zero. Hence, we must have $u'_N(x) = 0$ for all x . But the Clarke gradient at $x=0$ is overestimated at $[-1,1]$	8
4.1	When calculating the gradient of the critical point x , numerical approach will compute it via $g_x \approx \frac{f(x_2)-f(x_1)}{x_2-x_1}$, which may render significant errors. . . .	15
5.1	The Prototype of Automatic Measuring Framework	20
7.1	The plot for the solution (7.4) of PDE (7.3).	26
7.2	The $\ \cdot\ _1$ -norm of the gradient of the solution (7.4) of PDE (7.3).	26
7.3	Locating the regions	27
7.4	Plot of $f(x) = \text{abs}(\cos(x))$ in range of $[-10,2]$	28
7.5	Plot of a system of two equations $f(x)$ in range of $[-1,1]$	28
8.1	Planned Timeline	31
8.2	Actual Timeline	31

List of Tables

2.1	Table of Notation	4
3.1	Summarizing each method's feature	12
4.1	The features of Multi-dimensional Bisection and Interval Newton Method. .	15
7.1	Comparison of Lipschitz constant measured by CLEVER and Interval methods with differentiable sigmoidal activation function.	24
7.2	Comparison of Lipschitz constant measured by CLEVER and Interval methods with non-differentiable activation function - <i>ReLU</i>	24
7.3	Comparison of Lipschitz constant measured by CLEVER and Interval methods when the input is near boundaries.	25
7.4	Comparison of Lipschitz constant measured by CLEVER and Interval methods for PDE neural network.	26
8.1	Risks and Mitigated Methods	32

List of Abbreviations

BVP Boundary Value Problem

CLEVER Cross Lipschitz Extreme Value for nEtwork Robustness

MLE Maximum Likelihood Estimation

MLP Multilayer Perceptrons

NN Neural Network

Chapter 1

Introduction

1.1 Background

Robustness is the ability of a neural network (NN) to cope with noise in input data. A neural network is robust if tiny variations in inputs do not lead to significant changes in outputs [1]. Although neural networks have achieved exceptional performance in many machine learning tasks, such as image classification [2], object detection [3] and pose estimation [4], recent studies have highlighted their lack of robustness against adversarial examples [5]. Even the state of the art neural networks such as Faster R-CNN [6], ResNet [7] and FaceNet [8], exhibit robustness problems, in that a slight perturbation applied to the input will cause the neural network to produce unexpected outputs. One of the well-known examples is the one-pixel attack proposed by Su et al. [9]. Their study result shows that by modifying just one pixel, 67.97% of images in CIFAR-10 testing dataset can be disarrayed to incorrect target classes.

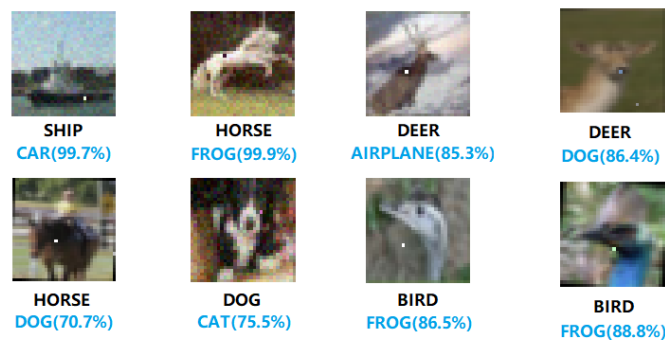


Figure 1.1: One Pixel Attack: the output of NNs can be easily altered by changing one pixel in the inputs

If these neural networks are used for safety-critical purposes, for instance, the criminal justice system, control of aeroplanes and road traffic, deviated results may have detrimental consequences. As neural networks are becoming a crucial method deployed in a wide variety of applications, including safety-critical systems, certifying the robustness of neural networks against adversarial perturbation has become an important research topic.

1.2 Motivation

Technically, the study of robustness in the neural network has three primary lines: studying the ways in which neural networks are vulnerable to attack; evaluating the robustness of neural networks and improving neural networks' robustness. In principle, these three challenges should be addressed separately. This indicates that the estimation of a neural networks robustness should not be affected by methods of attack, specifically when evaluating a new algorithm for robustness enhancement.

Unfortunately, at this stage, there is no reliable, standardized metric to evaluate the robustness of neural networks, which leads the existing approaches to using different scales to measure a target neural network's robustness. In particular, many existing studies regard

the attacking success rate and invalidation rate of the adversarial example based on a specific attack algorithm as robustness metrics [10]. As a result, the robustness evaluation result is intertwined with the attack algorithms. It will also cause the analysis to be restricted by the comprehensiveness of the attack. Therefore, it is worth investigating the factors that may affect the robustness of neural networks and *attack-agnostic* robustness evaluation methods.

The (local) Lipschitz constant of neural network has been determined to have a substantial impact on robustness estimation [11] and this estimating mechanism is attack-agnostic. Hence, several cutting-edge methods have taken advantages of it to assess robustness of neural networks [10]. However, estimating the exact Lipschitz constant is a proven NP-hard problem [12]. Therefore, the results gained from the majority of existing approaches (e.g. CLEVER [10]) are unverified and easily prone to errors. In addition, there exist many non-differentiable activation functions, such as *ReLU*, which cannot apply classical differentiation and chain rule to obtain the Lipschitz constant through back-propagation.

In this project, we propose a novel interval framework for validated estimation of Lipschitz constants in both differentiable and non-differentiable functions, and thereby evaluating the robustness of neural networks.

1.3 Aims and Objectives

The main aim of this project is to propose a novel framework to estimate the robustness of neural networks which is based on measuring the validated (local) Lipschitz constant. The framework should have the ability to analyse neural networks with *both differentiable and non-differentiable activation functions*. At the same time, the correctness of results should be guaranteed. We will also present an implementation of the framework and demonstrate how it may be used as a verifier for other robustness analysers. To achieve the aim, here are some specific objectives:

1. Scrutinizing existing studies to clarify factors that may lead to the lack of robustness;
2. Investigating previous methodologies used to evaluate robustness;
3. Proposing a novel method for robustness analysis based on interval arithmetic and bisection approach;
4. Extending the proposed method to identify the domains with the most insufficient robustness;
5. Extending the proposed method to take both differentiable (e.g., *Sigmoid*) and non-differentiable (e.g., *ReLU*) activation functions;
6. Comparing and analysing the performances of the proposed method and some of the established methods, e.g., CLEVER [10];
7. Summarizing the pros and cons of our main methodology and investigating other possible alternatives;
8. Designing and implementing a framework that can automatically measure robustness for MLPs with one-hidden layer (also known as, vanilla neural networks) with reasonable input dimensions.

1.4 Contributions

We highlight the main contribution of this project as follows. In Chapter 8.1, we will discuss the contributions of the project in more detail.

1. We proposed a novel interval framework for *certifying* robustness and the correctness of our method is proven by using the chain rule for Clarke gradient in interval arithmetic. To the best of our knowledge, our method is the first to combine interval computation with Clarke gradient to estimate the *validated* robustness of neural networks.
2. Since our bisection-based method adopts Clarke’s generalized gradient, our framework is able to analyse neural networks with *non-differentiable* activation functions, e.g., *ReLU*. Besides, because of the property of interval arithmetic, our method can *locate* regions of least robustness which can be used for enhancing robustness.
3. As the original goals of the project proposal had been achieved earlier than planned, we investigated another possible approach—i.e., interval Newton method—to improve the efficiency of our framework and make it applicable to larger networks. By adopting Clarke gradient, we relax the precondition of interval Newton method from continuously differentiable to Lipschitz constant. To the best of our knowledge, this is the first time that interval Newton method has been successfully used for the case of non-differentiable but Lipschitz continuous functions.

1.5 Dissertation Outline

This dissertation is structured into three major parts:

Chapter 1, 2, 3: These chapters address the current critical issues in certifying robustness and present the technical background for this project. At the end of these chapters, the reader will know **why** robustness evaluation has been reduced to Lipschitz constant estimation problem, **why** some cutting-edge methods are easily prone to errors, and **why** interval arithmetic and Clarke gradient are introduced in this project.

Chapter 4, 5, 6: In these chapters, we propose a novel interval framework and design the related algorithms as well as prototype. We also provide an introduction to the implementation of our framework. At the end of these chapters, the reader will know **how** our validated framework is designed and implemented with the aid of interval computation and Clarke’s generalized gradient.

Chapter 7, 8: These chapters present and discuss the experiment results to verify the validity of our proposed framework. We also detail contributions of this project and directions for future work. At the end of these chapters, the reader will know **what** this project has accomplished, **what** the impact and contributions of this project have made, and **what** the further directions are.

Chapter 2

Preliminaries

This chapter presents the theoretical preliminaries needed in this project which spanning four specific topics: Lipschitz Continuity, Clarke Gradient, Interval Arithmetic and Neural Network. Table 2.1 summarizes the notations used in the remaining of the dissertation. We start by defining the notions of special used terms in this project.

Table 2.1: Table of Notation

Notation	Definition	Notation	Definition
n	input vector's dimensionality	β_L	lower bound of minimum distortion
k	number of output classes	β_U	upper bound of minimum distortion
$\mathcal{F}$	the class of feed-forward networks	L_q	the (local) Lipschitz constant
$x_0 \in \mathbb{R}^n$	original input vector	q	the conjugate number of $q \in [1, \infty]$
$x_a \in \mathbb{R}^n$	adversarial example	$\partial f(\cdot)$	the Clarke gradient of function f
$c(\cdot)$	the predicted class	$B_p(x_0, R)$	hyper-ball with centre x_0 and radius R
$\delta \in \mathbb{R}^n$	adversarial perturbation	$ReLU(\cdot)$	$ReLU$ function
$\ \delta\ _p$	ℓ_p norm of distortion, $p \geq 1$	$Sigmoid(\cdot)$	$Sigmoid$ function
$\Delta_{p,min}$	minimum ℓ_p distortion of x_0	$abs(\cdot)$	absolute value function

Definition 1 (adversarial example)

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}^k$ be a k -class classification function, $x_0 \in \mathbb{R}^n$ be an input vector and the prediction of x_0 is $c(x_0) = \arg\max_{1 \leq i \leq k} f_i(x_0)$. A perturbed example x_a is the input vector x_0 with perturbation $\delta \in \mathbb{R}^n$ and $\ell_p^{(n)}$ -distortion Δ_p . The value x_a can be represented as $x_a = x_0 + \delta$ and $\ell_p^{(n)}$ -distortion Δ_p is defined as $\Delta_p = \|\delta\|_p^{(n)}$. If the predicted class of perturbed example $c(x_a) \neq c(x_0)$, then this perturbed example is an adversarial example.

Definition 2 (minimum adversarial distortion)

The minimum $\ell_p^{(n)}$ distortion of a input vector x_0 is denoted as $\Delta_{p,min}$. The minimum adversarial distortion $\Delta_{p,min}$ is defined as the smallest Δ_p in all the adversarial examples of x_0 .

Definition 3 (lower bound of minimum adversarial distortion)

The value β_L is denoted as a lower bound of $\Delta_{p,min}$ where $\beta_L \leq \Delta_{p,min}$. Any perturbed examples of x_0 with $\|\delta\|_p^{(n)} \leq \beta_L$ are not adversarial examples.

Definition 4 (upper bound of minimum adversarial distortion)

The value β_U is denoted as a upper bound of $\Delta_{p,min}$ where $\beta_U \geq \Delta_{p,min}$. The upper bound of $\Delta_{p,min}$ indicates that there exists an adversarial example in which $\ell_p^{(n)}$ -distortion $\|\delta\|_p^{(n)} \geq \beta_U$.

Definition 5 (the norm of distortion)

Let $\delta \in \mathbb{R}^n$ denote the distortion to the original example. For every $n \in \mathbb{N}$ and $p \in [1, \infty]$, let ℓ_p^n denote the Banach space $(\mathbb{R}^n, \|\cdot\|_p^{(n)})$, in which the norm $\|\cdot\|_p^{(n)}$ is defined as follows:

$$\|\delta\|_p^{(n)} := \begin{cases} (\sum_{i=1}^n |\delta_i|^p)^{1/p}, & \text{if } p \in [1, \infty), \\ \max\{|\delta_i| \mid 1 \leq i \leq n\}, & \text{if } p = \infty. \end{cases} \quad (2.1)$$

In addition, for every $p \in [1, \infty]$, the conjugate q can be defined as follows:

$$q := \begin{cases} 1, & \text{if } p = \infty, \\ \frac{p}{p-1}, & \text{if } p \in [1, \infty), \\ \infty, & \text{if } p = 1. \end{cases} \quad (2.2)$$

2.1 Lipschitz Continuity

Assume that (X, d_X) and (Y, d_Y) are two metric spaces, $f : X \rightarrow Y$, and $A \subseteq X$. A real number $L \geq 0$ is said to be a Lipschitz bound for f over A if:

$$\forall a, b \in A : d_Y(f(a), f(b)) \leq L d_X(a, b) \quad (2.3)$$

If the bound above exists, then the function f is said to be Lipschitz continuous over A and the smallest L is called the Lipschitz constant of f over A . Moreover, the following Lemma 1 reveals the relationship between Lipschitz constant and gradient norm.

Lemma 1 (Lipschitz constant and its relationship with gradient norm [13])

Assume that $S \subset \mathbb{R}^n$ is a convex bounded closed set and function $f(x)$, $f : S \rightarrow \mathbb{R}$, is a Lipschitz function with Lipschitz constant L_q if the following inequality holds for any $x, y \in S$:

$$|f(x) - f(y)| \leq L_q \|x - y\|_p \quad (2.4)$$

where $L_q = \sup\{\|\nabla f(x)\|_q : x \in S\}$, $\nabla f(x) = (\frac{\partial f(x)}{\partial x_1}, \dots, \frac{\partial f(x)}{\partial x_n})$ is the gradient of the function $f(x)$, and $\frac{1}{p} + \frac{1}{q} = 1$, $1 \leq p, q \leq \infty$.

In this project, the Lipschitz constant (L_q) is used to obtain a tight bound of the perturbation δ and due to the Lemma 1, the value of L_q can be gained from the gradient norm. However, it should be pointed out that though there is a close relationship between Lipschitz continuity and differentiability, they are not equal. For instance, the function $h : [-1, 1] \rightarrow \mathbb{R}$, defined by $h(x) = |x|$, is Lipschitz continuous, but not differentiable at 0. To summarize, we have the following inclusions for functions over compact subsets of $\mathbb{R}^n$:

$$\text{Continuously Differentiable} \subseteq \text{Lipschitz Continuous} \subseteq \text{Continuous}. \quad (2.5)$$

In fact, many commonly used activation functions in neural networks are Lipschitz continuous but not continuously differentiable. One example is function $ReLU$, defined by $\forall x \in \mathbb{R} : ReLU(x) := \max(0, x)$, is Lipschitz continuous everywhere but not differentiable at zero:

$$(ReLU)'(x) = \begin{cases} 1, & \text{if } x > 0, \\ 0, & \text{if } x < 0, \\ \text{undefined}, & \text{if } x = 0. \end{cases} \quad (2.6)$$

This means we cannot directly obtain L_q via calculating the norm of (classical) gradients based on Lemma 1. Fortunately, Clarke Gradient provides some solutions for this case.

2.2 Clarke Gradient

The generalized (Clarke) gradient is a replacement for the derivative and can be defined for functions over any Banach space. Assume that X is an open subset of $\mathbb{R}^n$, and $f : X \rightarrow \mathbb{R}$ is Lipschitz continuous over X and set $\mathcal{N}_f$ is defined as $\mathcal{N}_f := \{z \in X | f \text{ is not differentiable at } z\}$. The Clarke gradient $\partial f(x)$ of f at any $x \in X$ satisfies:

$$\partial f(x) = co\{\lim_{i \rightarrow \infty} f'(x_i) | \lim_{i \rightarrow \infty} x_i = x \wedge \forall i \in \mathbb{N} : x_i \notin \mathcal{N}_f\}. \quad (2.7)$$

Equation (2.7) indicates that taking any sequence $(x_i)_{i \in \mathbb{N}}$, which completely avoids the set $\mathcal{N}_f$ of points where f is not differentiable, converges to x , and for which the limit $\lim_{i \rightarrow \infty} f'(x_i)$ exists. The Clarke gradient is the convex hull of these limits. Back to the $ReLU$, although it is not differentiable at 0, one-sided limits of the derivative exist, i.e.:

$$\lim_{x \rightarrow 0^-} (ReLU)'(x) = 0 \quad \text{and} \quad \lim_{x \rightarrow 0^+} (ReLU)'(x) = 1. \quad (2.8)$$

Thus, based on 2.7 and 2.8, *ReLU*'s Clarke gradient at zero can be defined as:

$$\partial \text{ReLU}(0) = [0, 1] \quad (2.9)$$

Proposition 1

Let X be an open subset of ℓ_p^n and $f : X \rightarrow \mathbb{R}$ Lipschitz continuous over X with a Lipschitz bound L_q . Then, for every $x \in X$:

1. $\partial f(x)$ is a non-empty, convex, and compact subset of $\mathbb{R}^n$.
2. $\forall \xi \in \partial f(x) : \|\xi\|_q \leq L_q$.

Proof. See Proposition 2.1.2 in [14]. □

Lemma 1 relates the Lipschitz constant to the gradient norm. Typically, we access the norm of gradients through back-propagation, which is simply an efficient implementation of the familiar chain rule. However, the chain rule is only provably correct for functions that are compositions of continuously differentiable functions [15]. Thus, we need a chain rule for non-smooth functions based on Clarke gradient.

Proposition 2 (chain rule for non-smooth functions)

Assume that $X \subseteq \mathbb{R}^n$ is open, $h = (h_1, \dots, h_m) : X \rightarrow \mathbb{R}^m$ is Lipschitz continuous in a neighbourhood of $x \in X$, and $g : \mathbb{R}^m \rightarrow \mathbb{R}$ is Lipschitz continuous in a neighbourhood of $h(x) \in \mathbb{R}^m$. Then, $f := g \circ h$ is Lipschitz in a neighbourhood of x , and we have:

$$\partial f(x) \subseteq \bar{co} \left\{ \sum_{i=1}^m \alpha_i \zeta_i \mid \zeta_i \in \partial h_i(x) \wedge \alpha \in \partial g(h(x)) \right\}, \quad (2.10)$$

in which, $\bar{co}$ denotes the closed convex hull.

Proof. See Theorem 2.3.9 in [14]. □

Note that, unlike the chain rule for the classical derivative, the chain rule of (2.10) provides only a subset relation, which, in general, is not an equality. Our particular interval approach also adds a further layer of overestimation to the computations. In this project, we use n -dimensional boxes—also known as hyper-rectangles—to approximate the Clarke gradient. Although these hyper-rectangles may provides a set which is strictly larger than the Clarke gradient, soundness is retained and the implementation becomes simpler in the context of neural networks, because there is no need for extra operation.

2.3 Interval Arithmetic

As discussed in Section 2.2, the sub-differential of the *ReLU* function at zero is the closed interval $[0, 1]$. Therefore, using arbitrary-precision interval arithmetic with Clarke gradient to estimate Lipschitz constant of neural network becomes a natural way. Technically, an interval I (written $I = [\underline{I}, \bar{I}]$) is an ordered pair of real numbers $\underline{I}, \bar{I}$ with $\underline{I} \leq \bar{I}$. Further, it is natural to define the midpoint $m(I)$ and the width $w(I)$ of an interval as:

Definition 6 (midpoint and width of an interval)

For any interval $I = [\underline{I}, \bar{I}]$, the midpoint of I is defined by $m(I) := (\underline{I} + \bar{I})/2$ and the width of I is defined by $w(I) := (\bar{I} - \underline{I})$.

An interval can be viewed as a pair containing the upper and lower bound to an exact result which allows incorporating all the possible errors (e.g. floating-point round-off errors, truncations errors, etc.) into the result. The following presents some basic contents of interval arithmetic.

2.3.1 Interval Arithmetic Operations

Interval arithmetic defines a set of operations at intervals in the same way as classical arithmetic works on real numbers. Let $[a, b]$, $[c, d]$ are denoted to interval numbers. The basic interval arithmetic operations are as follows [16]:

1. Addition: $[a, b] + [c, d] = [a + c, b + d]$
2. Subtraction: $[a, b] - [c, d] = [a - d, b - c]$
3. Multiplication: $[a, b] \cdot [c, d] = [\min\{ac, ad, bc, bd\}, \max\{ac, ad, bc, bd\}]$
4. Division: $[a, b] \div [c, d] = [a, b] \cdot [1/d, 1/c]$ ($0 \notin [c, d]$)

In the definition of division above, we did not define the case that the divisor contained a zero. This situation can be handled by using extended interval arithmetic and it is necessary to define it in some cases, for instance, when a system has more than one root. Moreover, the results will be expressed as the union of two disjoint intervals instead of a single result interval. The complete definition of interval division can be extended as follows. Recall that:

$$[a, b] \div [c, d] = [a, b] \cdot [1/d, 1/c] \quad (2.11)$$

If $0 \notin [c, d]$, we adopt the basic interval division as above. If $0 \in [c, d]$, three cases will be taken into account which is based on extended interval arithmetic:

1. If $c = 0 < d$, then $[1/d, 1/c] = [1/d, +\infty)$.
2. If $c < 0 < d$, then $[1/d, 1/c] = (-\infty, 1/c] \cup [1/d, +\infty)$.
3. If $c < d = 0$, then $[1/d, 1/c] = (-\infty, 1/c]$.

We will not allow the case $c = d = 0$ here.

2.3.2 Finite Representations

In arbitrary precision interval arithmetic, every real number x is represented as the limit of a sequence $(I_i)_{i \in \mathbb{N}}$ of intervals with dyadic end-points, i.e., $x = \bigcap_{i \in \mathbb{N}} I_i$. Assume that we want to compute $\exp([1, 1]) = [e, e]$, in which e is the Euler number. As e is an irrational number, it does not have a finite binary expansion. For instance, in MPFI [17]—the arbitrary precision interval library that we use—when the precision is set at 10, the result of $\exp([1, 1])$ is given as $[2.718281828, 2.718281829]$.

Furthermore, as the precision of the end-points of each interval must be set first, inaccuracy is incurred even on simpler computations. For instance, if the precision (i.e., bit size) is set at n , then the result of $[0, 0] + [-2^{-2n}, 2^{-2n}]$ must be rounded outwards to $[-2^{-n}, 2^{-n}]$. This source of inaccuracy is inevitable and, to a large extent, harmless. The reason is that higher accuracy may be obtained by simply increasing the bit size of the representation of the end-points.

2.3.3 Dependency

Dependency problem, also known as wrapping-effect, is one of main flaw in interval computations which will cause overestimate [16]. By default, interval computations are agnostic of the dependencies between parameters. For instance, assume that $x \in [0, 1]$. By applying interval subtraction we obtain $(x - x) \in [-1, 1]$, which is quite an overestimation as $x - x = 0$, regardless of which interval x belongs to.

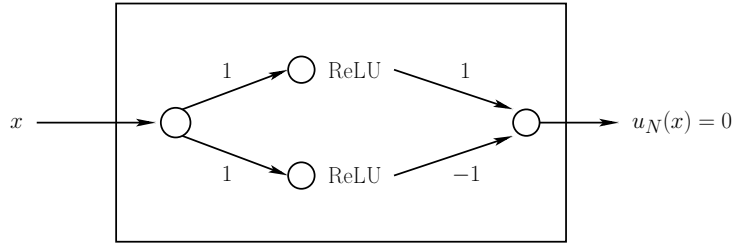


Figure 2.1: Dependency problem: The following network computes the constant function zero. Hence, we must have $u'_N(x)=0$ for all x . But the Clarke gradient at $x=0$ is overestimated at $[-1,1]$.

The dependency problem causes inaccuracies in estimations of the gradient as well. As an example, consider the neural network of Figure 2.1. This network computes the constant function $u_N(x)=0$. We estimate the Clarke gradient of u_N using the chain rule. As a result, at $x=0$, we obtain:

$$\partial u_N(0) = \partial \text{ReLU}(0) - \partial \text{ReLU}(0) = [0,1] - [0,1] = [-1,1]. \quad (2.12)$$

2.4 Neural Network

In this project, we focus on a specific class of networks:

Definition 7 (neural networks)

We let $\mathcal{F}$ denote the class of feed-forward networks with a single output neuron, and Lipschitz continuous activation functions.

The reason behind focusing on this class of neural networks is that, for each $N \in \mathcal{F}$, we can obtain an over-approximation of the Clarke gradient of u_N by a straightforward application of the chain rule of Proposition 2, as we will explain in Chapter 5.

Chapter 3

Related Work

A key issue in research on measuring robustness is that the robustness evaluation result is often intertwined with the attack algorithms. However, the attack algorithms easily overestimate the amount of perturbation needed to confuse the target neural network since the attack algorithms are not optimal and comprehensive. In other words, the methodology based on the attack algorithms gives the upper bound β_U , but the lower bound β_L is needed for safety guarantees [18] which ensures the neural network is robust to any perturbation with $\|\delta\|_p^{(d)} \leq \beta_L$. This chapter will include an overview of the studies in this field, focusing on attack-agnostic methods for estimating neural network robustness in both direct and indirect measurement approaches. Meanwhile, a retrospective review and evaluation of primary research methods in this area is presented to enhance understanding of this project.

3.1 Direct Robustness Measurement

Direct measurement methods will provide neural networks' robustness scores as an output. In the following, two main attack-agnostic methods for obtaining the neural networks' robustness will be discussed.

3.1.1 Theoretical Robustness Guarantees for Neural Networks

In 2017, Hein & Andriushchenko [19] provide a guaranteed lower bound for the robustness of one hidden layer MLP by using a local Lipschitz continuous condition. But it is challenging for neural networks with more than one hidden layer to derive their theory. Weng et al. [10] brought an extended formal robustness guarantees for a classifier based on Hein & Andriushchenko's theory. Their extended theory is universal, since Lipschitz continuity of the classification function is the only required assumption. Based on their work, a theoretical robustness guarantee on the lower bound of minimum distortion β_L for untargeted attack is given in Theorem 1.

Theorem 1 (Formal guarantee on β_L for untargeted attack)

Denoted L_q as the local Lipschitz constant of function $f_i(x) - f_j(x)$ at x_0 over a fixed ball $B_p(x_0, R) := \{x \in \mathbb{R}^d \mid \|x - x_0\|_p \leq R\}$. The formal guarantee on β_L can be defined as:

$$\|\delta\|_p \leq \min \left\{ \min_{j \neq i} \frac{f_i(x_0) - f_j(x_0)}{L_q}, R \right\} \quad (3.1)$$

3.1.2 CLEVER

CLEVER score [10] is a robustness metric based on estimating the local Lipschitz constant of a model, which is carried out using extreme value theory. CLEVER score is an attack-agnostic, computationally achievable for deep neural networks and is supported by Theorem 1 discussed above. Moreover, CLEVER is the first robustness metric independent from attack that can be applied to any neural network classifier and being widely used as the robustness metric in many adversarial robustness library like ART [20].

In CLEVER, measuring robustness is reduced to estimation of the Lipschitz constant, which is relied on extreme value theory. Algorithm 1 shows the steps of CLEVER methods.

Algorithm 1 Estimation of Lipschitz constant using CLEVER, extracted from [10].

Input:

$g: \ell_p^n \rightarrow \ell_\infty^1$, with the perturbation norm $p \in [1, \infty]$;
 $x_0 \in \mathbb{R}^n$: data sample; $r \geq 0$: radius of the neighbourhood;
 $N_b \in \mathbb{N}$: batch size; $N_s \in \mathbb{N}$: number of samples per batch;

Output: Approximation of the Lipschitz constant of g over $\overline{B}(x_0, r)$;

```

1:  $S \leftarrow \emptyset$ 
2:  $q \leftarrow$  conjugate of  $p$  as defined in (2.2)
3: for  $1 \leq i \leq N_b$  do
4:   for  $1 \leq k \leq N_s$  do
5:     randomly select a point  $x^{(i,k)} \in \overline{B}(x_0, r)$ 
6:     compute  $b_{ik} \leftarrow \|\nabla g(x^{(i,k)})\|_q$  via back-propagation
7:   end for
8:    $S \leftarrow S \cup \{\max\{b_{ik} \mid 1 \leq k \leq N_s\}\}$ 
9: end for
10:  $\hat{a}_W \leftarrow$  MLE of location parameter of reverse Weibull distribution on  $S$ 
11: return  $\hat{a}_W$ 

```

In a total of N_b iterations, CLEVER first generates N_s samples of $x^{(i,k)}$ over a fixed ball $\overline{B}(x_0, R)$ uniformly and independently in each batch. The value $\|\nabla g(x^{(i,k)})\|_q$ is computed via back-propagation and the maximum values of each batch will be stored in set S . Then, with samples S , CLEVER calculates a maximum likelihood estimation of reverse Weibull distribution parameters, and the estimate of Lipschitz constant is equal to the location estimate $\hat{a}_W$. The theory behind their estimation is Fisher-Tippett-Gnedenko Theorem [21] and reverse Weibull distribution can be defined by following:

$$\text{Reverse Weibull class: } G(y) = \begin{cases} \exp\left(-\left(\frac{a_W - y}{b_W}\right)^{c_W}\right), & \text{if } y < a_W \\ 1, & \text{if } y \geq a_W \end{cases} \quad (3.2)$$

where G is a non-degenerate distribution function, (a_n, b_n) is a sequence of pairs of real numbers and a_W is a finite right end-point.

3.2 Indirect Robustness Measurement

Neural networks with smaller norms of gradient have also been observed to be more robust [11]. This indicates that the neural network with larger Lipschitz constant has less robustness, since Lipschitz constant and norms of gradient are positively correlated, according to Lemma 1. Moreover, when robustness is directly estimated, both methods we discussed in section 3.1 have reduced measurement to estimating the (local) Lipschitz constant of a related function. Thus, bounding the (local) Lipschitz constants can be an indirect approach to certify the robustness of neural networks. In this section, we will introduce some state-of-the-art Lipschitz constant estimation algorithms.

3.2.1 Fast-Lip

Weng, et al. [22] propose a non-trivial algorithm, called Fast-Lip, to certify upper bound of the local Lipschitz constant. In their paper, all the neurons have been classified into three possible activation patterns—(i) always activated: $\mathcal{I}_k^+ := \{r \in [n_k] \mid \mathbf{u}_r^{(k)} \geq \mathbf{l}_r^{(k)} \geq 0\}$, (ii) always inactivated: $\mathcal{I}_k^- := \{r \in [n_k] \mid \mathbf{l}_r^{(k)} \leq \mathbf{u}_r^{(k)} \leq 0\}$, (iii) uncertain: $\mathcal{I}_k := \{r \in [n_k] \mid \mathbf{l}_r^{(k)} < 0 < \mathbf{u}_r^{(k)}\}$, where $\mathbf{u}_r^{(k)}$ and $\mathbf{l}_r^{(k)}$ are denoted to the upper and lower bound for the r -th neuron in the k -th layer. Their central idea is to exploit the three categories of activation patterns in

ReLU networks by rewriting the activation function $\sigma(\cdot)$ to the diagonal activation pattern matrix Λ which encodes the statuses of neurons r in k -th layer:

$$\Lambda_{r,r}^{(k)} = \begin{cases} 1, & \text{if } r \in \mathcal{I}_k^+, \\ 0, & \text{if } r \in \mathcal{I}_k^-, \\ 1 \text{ or } 0, & \text{if } r \in \mathcal{I}_k. \end{cases} \quad (3.3)$$

Then, they examine all the possible activation patterns and take the one (the worst-case) that results in the largest gradient norm. Hence, the highest Lipschitz constant is gained by the largest gradient norm.

3.2.2 LipSDP

LipSDP [23] is a convex optimization framework to compute guaranteed upper bound on the Lipschitz constant of neural networks. Their central idea is that regrading activation functions as gradients of convex potential functions. As a result, they meet certain properties defined by quadratic constraints which enables us to formulate the Lipschitz constant estimations problem as a semi-definite program (SDP). LipSDP considers both accurateness and scalability, however, it should be pointed out that it only works on ℓ_2 norm.

3.2.3 SeqLip

Scanman and Virmaux [12] have argued that exact Lipschitz computation is NP-hard, since this problem can be reduced to maximize a convex quadratic function on the hypercube which is a known NP-hard problem. Then, they propose a new algorithm—SeqLip which exploiting the relationships between singular vectors of consecutive layers to estimate Lipschitz constant:

$$\begin{aligned} L &\leq \max_{\forall i, \sigma_i \in [0,1]^{n_i}} \|M_K \text{diag}(\sigma_{K-1}) \cdots \text{diag}(\sigma_1) M_1\|_2 \\ &\leq \max_{\forall i, \sigma_i \in [0,1]^{n_i}} \left\| \sum_K V_K^\top \text{diag}(\sigma_K) U_{K-1} \sum_{k-1} V_{K-1}^\top \text{diag}(\sigma_{K-1}) \cdots \right\|_2 \\ &\leq \hat{L}_{SL} = \prod_{i=1}^{K-1} \max_{\sigma_i \in [0,1]^{n_i}} \left\| \sum_{i+1} V_{i+1}^\top \text{diag}(\sigma_{i+1}) U_i \sum_i \right\|_2 \end{aligned} \quad (3.4)$$

where L is the Lipschitz constant of the whole MLP, $\hat{L}_{SL}$ is denoted as the Lipschitz constant of the single layer, $M_i = U_i \sum_i V_i^\top$, $\tilde{\Sigma}_i = \Sigma_i$ if $i \in \{1, K\}$ and $\tilde{\Sigma}_i = \Sigma_i^{1/2}$ otherwise.

Furthermore, theoretical analysis of SeqLip is also provided:

Theorem 2 (theoretical analysis of SeqLip)

Let M_k be the matrix associated to the k -th linear layer, u_k (resp. v_k) its first left (resp. right) singular vector, and $r_k = s_{k,2}/s_{k,1}$ the ratio between its second and first singular values, then we have:

$$\hat{L}_{SL} \leq \hat{L}_{AL} \prod_{k=1}^{K-1} \sqrt{(1-r_k-r_{k+1}) \max_{\sigma_i \in [0,1]^{n_k}} \langle \sigma \cdot v_{k+1}, u_k \rangle^2 + r_k + r_{k+1} + r_k r_{k+1}} \quad (3.5)$$

where $\hat{L}_{AL}$ is denoted as the Lipschitz constant gained from automatic differentiation algorithms. Moreover, if the ratios r_k are negligible, then:

$$\hat{L}_{SL} \leq \hat{L}_{AL} \prod_{k=1}^{K-1} \left| \max_{\sigma_i \in [0,1]^{n_k}} \langle \sigma \cdot v_{k+1}, u_k \rangle \right| \quad (3.6)$$

and

$$\hat{L}_{SL} \approx \frac{\hat{L}_{AL}}{\pi^{K-1}} \quad (3.7)$$

3.3 Discussion

All the above methods are essentially estimating Lipschitz constant. Specifically, according to Theorem 1, the lower bound β_L of ℓ_p minimum distortion can be obtained by $\beta_L = \min_{i \neq j} \frac{f_i(x_0) - f_j(x_0)}{L_q}$. Therefore, evaluating robustness is reduced to a (local) Lipschitz constant estimation problem. However, it has been proved that estimation of exact Lipschitz constant is a NP-hard problem [12]. Thereby, the methods discussed above are not optimized and have their pros and cons. Table 3.1 summarizes each method’s features below.

Table 3.1: Summarizing each method’s feature

Methods	Norm	Support Theory	Pros	Cons
CLEVER	$\ell_1, \ell_2, \ell_\infty$	Extreme Value Theory	Fast, Scalable	Based on statistic, may lose accuracy
Fast-Lip	$\ell_1, \ell_2, \ell_\infty$	Activation Pattern	Easy implementation	Not really fast, need speed-up tricks
LipSDP	ℓ_2	Semi-definite Program	Sound theory	Can only have accuracy or scalability
SeqLip	ℓ_2	Linear Computation Graph	Sound theory	Need heuristics trick for large NNs

Moreover, as the difficulty of computing exact Lipschitz constant value, a useful estimate should be a tight upper bound of the Lipschitz constant. The reason is that, a lower bound of the Lipschitz constant would result in false negatives which are more serious than false positives and a very loose upper bound would result in too many false positives, which renders the estimate useless.

In this project, we use CLEVER as the main reference statistical algorithm and demonstrate how our interval-based framework can be used to estimate the tight upper bound of the Lipschitz constant as well as verify statistical methods. As discussed in Section 3.1.2, CLEVER are supported by a rich statistical theory and implemented easily in practice, but this method is prone to inaccuracies.

From a computational angle, the crucial step occurs in line 10 of Algorithm 1. The accuracy of the estimation of the location parameter relies on the numbers N_b and N_s . In other words, some estimate of how many batches and samples per each batch are sufficient for a given accuracy is required. The threshold for N_b and N_s could depend on a number of factors, e.g., the dimension of the input space, the location of x_0 and its proximity to the boundaries of various classes, the type of activation function(s), number of layers in the network, and many other factors. On the implementation side, the accuracy is further affected by floating-point inaccuracies.

Gradient masking [18] is one strong piece of evidences which proves CLEVER overestimates the adversarial perturbation size. In what follows, we also demonstrate that these inaccuracies do indeed affect the result returned by Algorithm 1.

Chapter 4

Methodology

4.1 Interval Methodologies

In this project, we take the indirect approach—measuring Lipschitz constant to estimate the robustness of the neural network. In order to obtain the tight upper bound of Lipschitz constant in the input domain, we introduce two interval methodologies:

1. multi-dimensional bisection;
2. interval newton method.

In what follows, a discussion and comparison of these two methods will be presented as well.

4.1.1 Multi-dimensional Bisection

Multi-dimensional bisection is a straightforward method used in arbitrary precision interval arithmetic to certify the tight upper bound of L_q . In this project, we use n -dimensional boxes B as the input. Then, bisecting each box into two smaller boxes in each iteration. Therefore after each bisection, 2^n (smaller) boxes are created, where n is the dimension of the input space. As such, the time complexity of bisection is exponential which limits this method's speed and scalability. Because of this significant flaw, bisection is used in low dimensional cases in this project. However, it should be pointed out that bisection method is sound, reliable and always gives results—though the results may hold numerous overlapping boxes in the end. Another advantage of bisection method is that it can provide the possibility to isolate the boxes (domains) which cause highest Lipschitz constant. In other words, we can obtain the domains where lack the most robustness by using bisection method.

4.1.2 Interval Newton Methods

Since the main drawback of bisection is inefficiency, another efficient method considered in this project is interval Newton methods. There are two operator we take, the interval Newton operator and Krawczyk operator which is based on the Newton-Raphson iteration. In general, the Krawczyk operator shares properties with the Newton operator, but with less computational costs [24]. Hence, in this project, we take Newton operator for one-dimensional cases and Krawczyk operator for multi-dimensional input vectors.

Theorem 3 (Newton operator in one dimension [16])

Assume f is a real-valued function of a real variable x , and suppose that f is Lipschitz continuous. Let $F'(X)$ be an inclusion monotonic interval extension of $f'(x)$, defined on interval vectors $X \subseteq D$. Then, we have:

$$X^{(k+1)} = X^{(k)} \cap N(X^{(k)}) \quad (k=0,1,2,\dots), \quad (4.1)$$

where

$$N(X) = m(X) - f(m(X)) / F'(X). \quad (4.2)$$

The interval $X^{(k)}$ form a nested sequence converging to x if $0 \notin F'(X^{(0)})$.

Proof. See Theorem 8.1 in [16]. □

Theorem 4 (Krawczyk operator [16])

Assume $f(x)$ is a finite system of nonlinear equations and $F(X)$ is the isotonic interval extension for f . Let $y = m(X)$ be a real vector contained in the interval vector $X \in D$. Let Y be a nonsingular real matrix approximating the inverse of the real Jacobian matrix $F'(m(X))$ with elements $F'(m(X))_{ij} = \partial f_i(x)/\partial x_j$ at $x = m(X)$. I denotes the identity matrix. Then, define $K(X)$ by

$$K(X) = y - Y f(y) + \{I - Y F'(X)\}(X - y) \quad (4.3)$$

If $K(X) \in X$, then $F(X)$ has a solution in X . It is also in $K(X)$.

Proof. See Theorem 8.2 in [16]. □

When it converges, the interval Newton method has a significantly higher convergence rate compared with bisection.

To be clear, there is a simple example about how interval Newton method is used in this project. Assume that $N \in \mathcal{F}$ is a feedforward network with a single output neuron, $S = \prod_{1 \leq i \leq n} I_i \subseteq \mathbb{R}^n$ is a hyper-rectangle, together with an interval function f over S and let $u_N : S \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$, for some $n \in \mathbb{N}$. The function u_N may be written in closed form in terms of the weights, biases, and the activation functions of N . Using the chain rule of Proposition 2, we obtain a closed form expression for a superset of $\partial u_N(x)$, for every $x \in S$. As the norm of distortion we chosen in this project is L_∞ -norm, the gradient norm used in computing Lipschitz constant should be L_1 -norm where we have $\|\partial u_N(x)\|_1 = \sum_{i=1}^n |x_i|$.

Thus, we inevitably have to take absolute value function which is Lipschitz continuous but not continuously differentiable. That is the main reason why we need Lipschitz interval Newton method rather than the classical continuously differentiable condition. Moreover, when linking interval Newton method with neural networks, this problem can be gradually abstracted into two stages:

Stage 1. Abstracting the neural network into a multivariable function.

Stage 2. Abstracting the gradient norms of this multivariable function into a system of nonlinear and nonsmooth equations.

This means that interval Newton method must firstly be proved working on a system of nonlinear and nonsmooth equations (Stage 2), before it can be extended to neural networks (Stage 1). However, one precondition in the available literature of using interval Newton method is that the functions must be continuously differentiable whereas as mentioned previously, we need the interval Newton method to operate on Lipschitz continuous functions. Whether the precondition can be relaxed from continuously differentiable to Lipschitz continuous is still an unknown problem. Due to the restricted time, in this project, we will focus on Stage 2 and attempt to relax the precondition to Lipschitz continuous by replacing the classical gradient with the Clarke gradient.

4.1.3 Discussion

This crucial idea behind our method is to acquire the highest local Lipschitz constant in the input domain space via interval arithmetic. In this project, we provide two possible interval approaches for searching the solution—multi-dimensional bisection and interval Newton method. Multi-dimensional bisection is an universal method that it can be used in any situations and always yields results—no matter they are acceptable or not. But, its drawbacks are also obvious and one is low rate of convergence—linear convergence which makes this method almost impossible to be applied in high dimensional datasets if no speed-up tricks are used. As a result, we need an efficient method and the interval Newton method is a proper candidate. The interval Newton method has quadratic convergence, which has the potential to be extended to high dimensions. However, according to current study,

interval Newton method cannot guarantee to converge and cannot be used for nonsmooth functions.

We choose multi-dimensional bisection to be the main methodology for this project, since it more universal and can guarantee to give outcomes. Then, as the original project was completed ahead of schedule, we add interval Newton method as the bonus methodology. Moreover, because it is unknown whether interval Newton method can be applied to Lipschitz continuous but not continuously differentiable functions, we will focus on attempting to implement interval Newton to a system of nonlinear and nonsmooth equations (Stage 2). Table 4.1 summarizes the features of bisection and interval Newton method.

Table 4.1: The features of Multi-dimensional Bisection and Interval Newton Method.

Methods	Universality	Speed	Scalability	Rate of Convergence	Role in the Project
Multi-dimensional Bisection	Yes	Slow	Low	Linear	Main methodology
Interval Newton Method	May not converge	Fast	High	Quadratic	Bonus methodology

4.2 Symbolic Computation

The computations in the back-propagation method can depend on numerical differentiation, which includes numbers directly, or symbolic differentiation, in which the variables are symbols. In general, the numerical computation has simpler and faster implementation than symbolic computation, but this comes at the cost of accuracy. Figure 4.1 illustrates how numerical differentiation may render errors. In contrast, the symbolic method is more accurate whereas it may result in complex and redundant expressions.

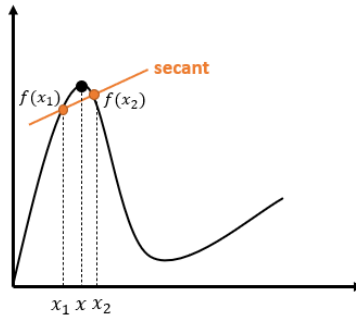


Figure 4.1: When calculating the gradient of the critical point x , numerical approach will compute it via $g_x \approx \frac{f(x_2) - f(x_1)}{x_2 - x_1}$, which may render significant errors.

Actually, because of the efficient and numerically stable, the commonly used machine learning libraries, like PyTorch [25], take automatic differentiation (i.e., autodiff) which is a method between numerical and symbolic. Although the autodiff has numerous advantages, it also induces significant numerical error [26]. Since the primary aim in our method is to guarantee the result's accuracy, we will adopt the symbolic computation in the process of back-propagation. [27] provides a symbolic expression for the derivative of the MLP with one hidden layer:

$$\frac{\partial^k N}{\partial x_j^k} = \sum_{i=1}^H v_i w_{ij}^k \sigma_i^{(k)} \left(\sum_{j=1}^N w_{ij} x_j + u_i \right) \quad (4.4)$$

in which, x_i is one element of input vector; N denotes the output of the NN; H is the number of activation units in the hidden layer; v_i represents the weight from the hidden unit i to the output; w_{ij} indicates the weight from the input unit j to the hidden unit i ; u_i denotes the bias of hidden unit i and $\sigma^{(k)}$ is the k^{th} order derivative of the activation function σ .

4.3 Project Methodologies

This section presents three methodologies I used in the project: methodology for development, testing methodology, and methodology for version control.

4.3.1 Development Methodology

The Agile-Waterfall hybrid methodology is used to structure the project. Planning, design, and requirements definition will be completed with Waterfall. However, as this project is research-driven, most of the following tasks rely on the outcomes of the previous tasks. Therefore, each task will be developed and tested in short sprints by using Agile.

4.3.2 Testing Methodology

Unit testing is introduced in this project to verify that each unit of the framework performs as intended. There are some advantages of using unit testing. For example, unit testing boosts confidence in code maintenance. Furthermore, codes must be flexible in order to allow for unit testing. This means the codes can be reused more easily.

4.3.3 Version Control Methodology

This project incorporates Git as the methodology for version control. Git is the version control system that is most widely used and will trace any change made in this project. By using git, a log will be generated that the project can revert to specific past version if there has any mistake. Moreover, git could also provide evidence for revision in future.

Chapter 5

Design

The design of this project is presented in this chapter which consists of three aspects. The first is the bisection-based maximization algorithm to obtain the tight upper bound of local Lipschitz constant. Then, two algorithms based on interval Newton method for a system of nonlinear equations are introduced. Last, the prototype of implemented interval framework is presented.

5.1 Bisection-based Maximization Algorithm

Assume that $N \in \mathcal{F}$ is a feedforward network with a single output neuron, and let $u_N : S \subseteq \mathbb{R}^n \rightarrow \mathbb{R}$, for some $n \in \mathbb{N}$. The function u_N may be written in closed form in terms of the weights, biases, and the activation functions of N . Using the chain rule of Proposition 2, we obtain a closed form expression for a superset of $\partial u_N(x)$, for every $x \in S$. As we use interval arithmetic, all the approximations are hyper-rectangles in $\mathbb{R}^n$. Hence, there is no need for computation of closed convex hull as hyper-rectangles are closed and convex. We denote these hyper-rectangles by $\partial \hat{u}_N(x)$. Computing the ℓ_1^n -norm of such a hyper-rectangle results in an interval $\|\partial \hat{u}_N(x)\|_1$.

Assume that $S' \subseteq S$ is a hyper-rectangle in $\mathbb{R}^n$. If we pass S' , and the interval function $x \mapsto \|\partial \hat{u}_N(x)\|_1$ over S' , to Algorithm 2, then we will obtain an enclosure of the maximum Lipschitz norm, together with an enclosure of the *maximum set*, i.e., the set of points where the maximum Lipschitz norm is attained.

Algorithm 2 also requires three other input parameters, i.e., m_i , m_b , and δ . These are crucial for making sure that the algorithm stops below a reasonable threshold. In line 7, we discard all those boxes in B which cannot possibly contain the maximum value. How many boxes are discarded depends on a number of factors, most importantly, whether the function f has a flat section or not. The condition $|B| \geq m_b$ in line 11 is necessary because in many cases flat sections exist and the number of elements of B grows exponentially with each iteration.

There are two elements returned by Algorithm 2: the tight interval upper bound of Lipschitz constant I_B and a set of interval boxes B (domains) that contain the regions of the highest local Lipschitz constant. The value of I_B can be regarded as a metric of the neural network's robustness as discussed in the Chapter 3. The set of interval boxes B can indicate the domains where lack the most robustness. Therefore, we can aim at these weak domains to launch attacks or prevent attacks.

5.2 Interval Newton-based Algorithm

In this section, we consider finite systems of nonlinear equations which are Lipschitz continuous:

$$\begin{aligned} f_1(x_1, \dots, x_n) &= 0, \\ &\vdots \\ f_n(x_1, \dots, x_n) &= 0. \end{aligned} \tag{5.1}$$

Algorithm 2 Estimation of the maximum value and the maximum set of an interval function.

Input:

A hyper-rectangle $S = \prod_{1 \leq i \leq n} I_i \subseteq \mathbb{R}^n$, together with an interval function f over S ;
 $m_i \in \mathbb{N}$: maximum iterations;
 $m_b \in \mathbb{N}$: maximum number of boxes;
 $\delta > 0$: a positive dyadic number indicating maximum width.

Output: An interval enclosure of the maximum value of f over S and an enclosure of the maximum set

```

1:  $B_1 \leftarrow S$  // initialize the set of hyper-rectangles
2:  $i \leftarrow 0$  // number of iterations
3: repeat
4:    $B \leftarrow B_1$ 
5:    $L_B := \{f(b) \mid b \in B\}$ 
6:    $z \leftarrow \max\{\underline{I} \mid I \in L_B\}$  // the maximum left end point
7:    $B \leftarrow \{b \in B \mid z \in f(b)\}$  // retain only those boxes that could contain maximum points
8:    $I_B := \bigcup \{f(b) \mid b \in B\}$  // take the union of all the overlapping intervals
9:    $B_1 \leftarrow \text{bisect}(B)$  // bisect all boxes in  $B$  for the next iteration
10:   $i \leftarrow i + 1$ 
11: until ( $i \geq m_i$ ) or ( $|B| \geq m_b$ ) or ( $w(I_B) \leq \delta$ )
12: return  $I_B$  and  $B$ 

```

and we may write in vector notation as:

$$f(x) = 0 \tag{5.2}$$

When the input vector only has one dimension ($n = 1$), we take interval Newton method to find solutions of $f(\cdot)$ via Algorithm 3. In order to find the roots of the function f , we pass a hyper-rectangle S in $\mathbb{R}$, a Lipschitz continuous function f and its derivative interval extension F' to Algorithm 3. Moreover, we also input $m_i \in \mathbb{N}$ to stop the algorithm. The crucial step in this algorithm is line 9. Instead of using classical derivative, we adopt Clarke gradient in $F'(\cdot)$ to solve the problems caused by nonsmooth functions. Krawczyk operator will be introduced in the high dimensional cases. Compared with interval Newton method, the key difference is at Algorithm 4 line 9. Moreover, we also replace the classical gradient with the Clarke gradient.

5.3 Prototype of Automatic Measuring Framework

In addition to the algorithms above, designing a automatic measuring framework is one of objectives as well. According to the discussion in section 4.1.3, multi-dimensions bisection will be chosen as the main methodology. Hence, the interval robustness measuring framework is based on multi-dimensions bisection method and can automatically produce the heightist local Lipschitz constant for the input model which currently restricted to one hidden layer neural network. Figure 5.1 below shows an intuitive prototype of this system.

This framework uses Python as a wrapper language to gain the parameters of the MLP and the Lipschitz constant via CLEVER method if require. Then, the parameters will be passed to code built by C++ to obtain the interval Lipschitz constant and locate the regions of least robustness. The implementation details of this framework will be described in Chapter 6.

Algorithm 3 Finding the solutions of a nonlinear equation via interval Newton operator.

Input:

S : an one-dimensional hyper-rectangle;
 f : a Lipschitz continuous function;
 $F'(\cdot)$: the interval extension of $f'(\cdot)$;
 $m_i \in \mathbb{N}$: maximum iterations.

Output: The set of solution domain(s) for this function f : $B_{results}$.

```

1:  $B \leftarrow S$  // initialize the set of hyper-rectangles
2:  $i \leftarrow 0$  // number of iterations
3: repeat
4:    $B_{results} \leftarrow \emptyset$ 
5:   for  $b$  in  $B$  do
6:      $X \leftarrow b$ 
7:     //  $N(X)$  may have more than one hyper-rectangle in it.
8:     //  $F'(\cdot)$  is based on Clarke gradient
9:      $N(X) = m(X) - f(m(X)) \div F'(X)$ 
10:     $B_1 := \{T_b \mid T_b = b \cap X, T_b \neq \emptyset, b \in N(X)\}$ 
11:     $B_{results}$  appends  $B_1$ 
12:  end for
13:   $B \leftarrow B_{results}$ 
14:   $i \leftarrow i + 1$ 
15: until ( $i \geq m_i$ )
16: return  $B_{results}$ 

```

Algorithm 4 Finding the solutions of a system of nonlinear equations via Krawczyk operator.

Input:

S : a multi-dimensional hyper-rectangle;
 f : a Lipschitz continuous function;
 $F'(\cdot)$: the interval extension of $f'(\cdot)$;
 $m_i \in \mathbb{N}$: maximum iterations.

Output: The set of solution domain(s) for this function f : $B_{results}$.

```

1:  $B \leftarrow S$  // initialize the set of hyper-rectangles
2:  $i \leftarrow 0$  // number of iterations
3: repeat
4:    $B_{results} \leftarrow \emptyset$ 
5:   for  $b$  in  $B$  do
6:      $X \leftarrow b$ 
7:     //  $K(X)$  may have more than one hyper-rectangle in it.
8:     //  $F'(\cdot)$  is based on Clarke gradient;  $I$  denotes the identity matrix.
9:      $K(X) = m(X) - F'(m(X)) \cdot f(m(X)) + \{I - F'(m(X)) \cdot F'(X)\}(X - m(X))$ 
10:     $B_1 := \{T_b \mid T_b = b \cap X, T_b \neq \emptyset, b \in K(X)\}$ 
11:     $B_{results}$  appends  $B_1$ 
12:  end for
13:   $B \leftarrow B_{results}$ 
14:   $i \leftarrow i + 1$ 
15: until ( $i \geq m_i$ )
16: return  $B_{results}$ 

```

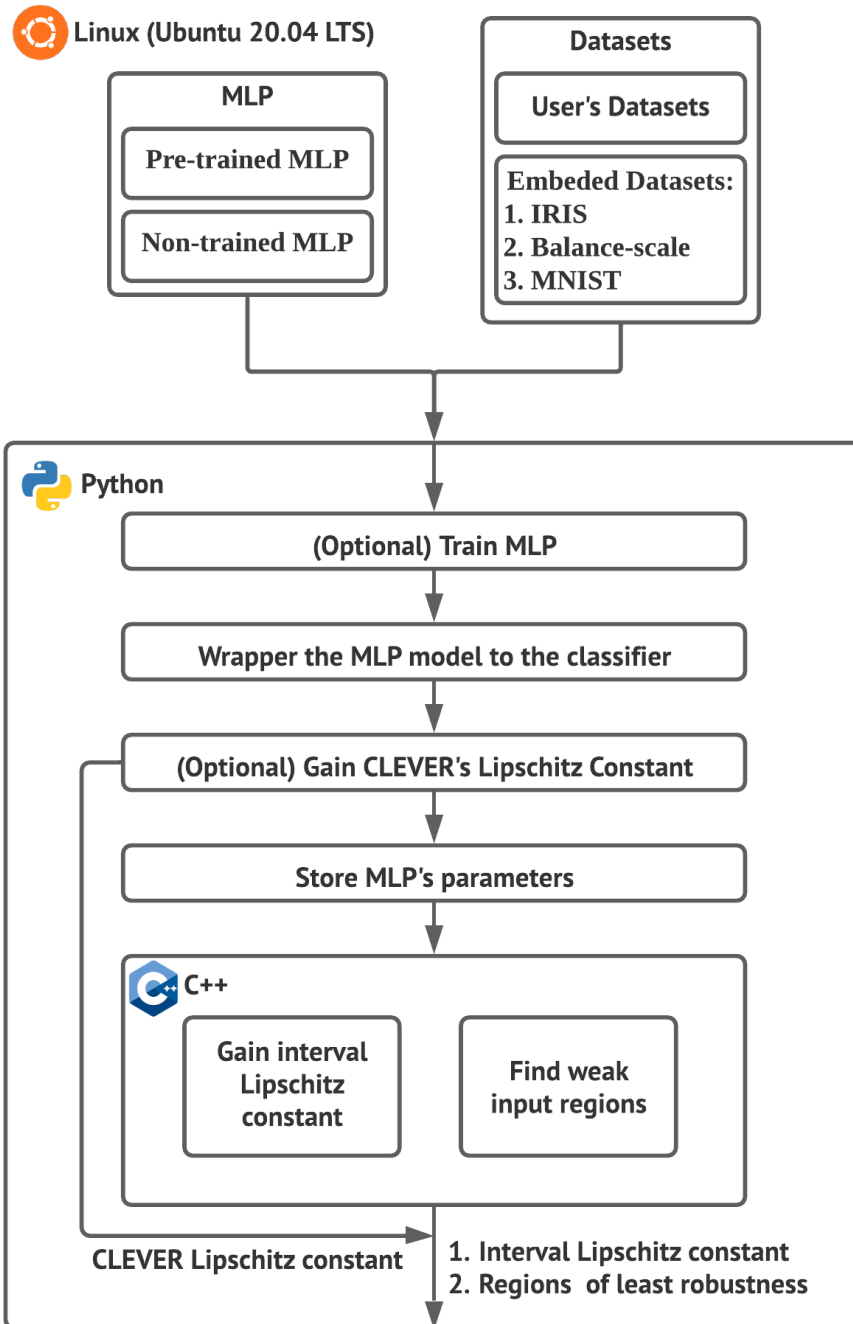


Figure 5.1: The Prototype of Automatic Measuring Framework

Chapter 6

Implementation

The implementation of this project are specified in this chapter and is divided into three parts. The choices of programming languages, libraries and the running environment is discussed first. After that the implementation of automatic measuring framework designed in Section 5.3 is presented. Finally, this chapter covers the definition of the list of nonlinear functions in both smooth and non-smooth categories.

6.1 Environment, Languages, and Libraries

This section will discuss the programming languages, libraries and system environment used in this project in detail which also has demonstrated in Figure 5.1. The operating system—Linux (Ubuntu 20.04 LTS) is set to the running environment of this project in order to be compatible with the tools and libraries required by each task.

Two programming languages will be used in this project: C++ and Python. C++ is a well structured and written language with high running speed. This project is mostly based on the interval method, but this method has a significant flaw: its computations are more complex than machine level floating-point computations, which leads to a sluggish operating speed. Compared to other languages like Python, the programming coded in C++ naturally has a higher running speed. Hence, C++ is more suitable for the system based on interval methods. In addition, C++ has several powerful libraries that are also well written and can accommodate high-precision calculations. Python will be used as a wrapper language in this project to string key components together in the framework. In addition, Python is also used to calculate CLEVER score and train neural networks because it has several strong frameworks like Pytorch to support machine learning tasks.

The main library used to support interval calculation is MPFI [17] (Multiple Precision Floating-point Interval library). At the same time, Boost Library [28] offers a user-friendly wrapper for MPFI in C++ that allows the easy use of MPFI in this project. For the machine learning tasks, the Pytorch [25] framework has been chosen because it has powerful functions which is easy to program and can be customised for each model's parameters.

6.2 Implementation of Automatic Measuring System

The main procedure is described in Section 5.3 and measuring Algorithm 2 has been introduced in Section 5.1. This section present some implementation details. First one is the method of combining codes in Python and C++. As Python is the wrapper language, the C++ code will be compiled into external Python library via pybind11 [29]—a lightweight header-only library. Hence, the interval Lipschitz constant of the neural network can be calculated in C++ codes by directly calling in the Python environment. Another detail of the automatic measuring framework is that users can customize their requirements. For instance, users can choose the embedded datasets or upload their own datasets. Moreover, users can decide whether the interval Lipschitz constant will be compared with CLEVER results as well. In addition, for each function in the project, we have apply the Unit Test to ensure the accuracy and quality of the code and Git has been employed for version control.

6.3 List of Functions Used in the Project

Both smooth and non-smooth instance functions have been applied in this project. In particular, the *Sigmoid*($\cdot$) is chosen to be the example of differentiable activation function and for non-differentiable functions, two instances—*ReLU*($\cdot$) and absolute (*abs*($\cdot$)) function are introduced. It should be noted that those non-smooth function are Lipschitz continuous. Thus, we could use the (2.7) to define the derivation of the function and apply the chain rule in the computation process which is based on the Proposition 2.10. All the preliminaries has introduced in Chapter 2.

- (I) *Sigmoid*($\cdot$) is the instance of the differentiable (smooth) activation functions. The Sigmoid function itself for interval input $[\underline{x}, \bar{x}]$ is defined as:

$$Sigmoid([\underline{x}, \bar{x}]) = \frac{1}{1 + e^{-[\underline{x}, \bar{x}]}} \quad (6.1)$$

Since *Sigmoid* function is continuous differentiable, the first derivative of Sigmoid can be represented following:

$$\partial Sigmoid([\underline{x}, \bar{x}]) = Sigmoid([\underline{x}, \bar{x}]) * (1 - Sigmoid([\underline{x}, \bar{x}])) \quad (6.2)$$

- (II) *ReLU*($\cdot$) is one example of non-differentiable activation functions. The *ReLU* function itself for interval input $[\underline{x}, \bar{x}]$ is defined as:

$$ReLU([\underline{x}, \bar{x}]) = \begin{cases} [\underline{x}, \bar{x}], & \text{if } \underline{x} > 0, \\ [0, 0], & \text{if } \bar{x} < 0, \\ [0, \bar{x}], & \text{else.} \end{cases} \quad (6.3)$$

ReLU is Lipschitz continuous everywhere but not differentiable at zero. According to the Proposition 1, the Clarke gradient of *ReLU* can be defined as:

$$\partial ReLU([\underline{x}, \bar{x}]) = \begin{cases} [1, 1], & \text{if } \underline{x} > 0, \\ [0, 0], & \text{if } \bar{x} < 0, \\ [0, 1], & \text{else.} \end{cases} \quad (6.4)$$

- (III) *abs*($\cdot$) is also an example of non-differentiable functions. The ℓ_1 gradients norm of input vector $X \in \mathbb{R}^n$ can be computed via $\|X\|_1 = \sum_{i=1}^n |X_i|$, where $n \in \mathbb{N}$. Thus, the Clarke gradient of absolute function is needed when applying the in interval Newton method. The absolute function itself for interval input $[\underline{x}, \bar{x}]$ is presented as:

$$abs([\underline{x}, \bar{x}]) = \begin{cases} [\underline{x}, \bar{x}], & \text{if } \underline{x} > 0, \\ [-\bar{x}, -\underline{x}], & \text{if } \bar{x} < 0, \\ [0, \max(-\underline{x}, \bar{x})], & \text{else.} \end{cases} \quad (6.5)$$

Similarly, absolute function is Lipschitz continuous everywhere but not differentiable at zero. According to the Proposition 1, the Clarke gradient of *abs* can be represented as:

$$\partial abs([\underline{x}, \bar{x}]) = \begin{cases} [1, 1], & \text{if } \underline{x} > 0, \\ [-1, -1], & \text{if } \bar{x} < 0, \\ [-1, 1], & \text{else.} \end{cases} \quad (6.6)$$

Chapter 7

Evaluation

7.1 Experiment Setup

We have implemented our framework using the MPFI [17] library. In this chapter, we firstly present the results of some of our experiments, where we compare the performance of our bisection-based algorithm with that of CLEVER. Furthermore, as opposed to (say) CLEVER, our interval method provides the location of the points with least robustness. In addition, for interval Newton method, we demonstrate it can be implemented for a system if nonlinear and nonsmooth function (Stage 2 in Section 4.1.2).

We perform our experiments on three different data sets: IRIS¹, Balance-scale², and MNIST³. The first two are instances of low-dimensional data sets, each having four input attributes and three output classes. MNIST, which is a handwritten digit dataset, with 784 input attributes and 10 output categories, is an example of a high dimensional data set.

For each dataset, we train two sets of networks, one with the differentiable sigmoid activation function, and another with the non-differentiable *ReLU* activation function. We will also perform some experiments on neural networks trained for solving PDE. These networks are not trained on any dataset, and the activation function is the sigmoidal function.

The accuracy of the CLEVER algorithm is measured by the relative error of its result v_C compared with that of the interval results $v = [\underline{v}, \bar{v}]$ from our algorithm. Specifically, if $v_C \leq \underline{v}$, which happens in most experiments, the relative error will be measured by:

$$\text{Relative error} := \frac{|v_C - \underline{v}|}{\underline{v}} \quad (7.1)$$

while if $v_C \leq \underline{v}$, then $\underline{v}$ will be replaced by $\bar{v}$ in (7.1). If $v_C \in v$ —which does not happen in our experiments—then we would take the relative error to be 0.

We should also point out that, as CLEVER is a statistical algorithm, we had to run each experiments several times and take the average. The columns in the Tables 7.1, 7.2, 7.3, and 7.4 for CLEVER record the average score of 10 executions of the algorithm.

Since the interval Newton based algorithm has not been applied in neural networks (Stage 1 in Section 4.1.2), our current experiments focus on investigating whether this method can be applied into a system of nonlinear and nonsmooth functions (Stage 2 in Section 4.1.2). In particular, we choose the function $f(x) = \text{abs}(\cos(x))$ as the example of one dimensional and for multi-dimensional, we take the a system of two equations below:

$$\begin{cases} \text{abs}(\sin(x_1)) + 2 \cdot x_2 = 0, \\ x_1 + x_2 = 0. \end{cases} \quad (7.2)$$

¹<https://archive.ics.uci.edu/ml/datasets/iris>

²<https://archive.ics.uci.edu/ml/datasets/balance+scale>

³<http://yann.lecun.com/exdb/mnist/>

7.2 Differentiable Activation Function

For IRIS and Balance-scale datasets, we have trained neural networks with the sigmoidal activation function, each having 10 hidden neurons in their only hidden layer. The regions where the Lipschitz constant are measured are of centre-radius form. The centres have been chosen randomly in this experiment. In particular, for IRIS, the centre $c = (5.8, 4.0, 1.2, 0.2)$ and for Balance-scale, the centre $c = (5.0, 3.0, 2.0, 3.0)$ have been chosen. Table 7.1 summarizes the results of this experiment. The radii are given in the table.

The results clearly show that CLEVER underestimates the Lipschitz constant, leading to false negatives. While the relative errors are generally not high, still a certified algorithm must not lead to false negatives.

Table 7.1: Comparison of Lipschitz constant measured by CLEVER and Interval methods with differentiable sigmoidal activation function.

Dataset	Accuracy(%)	Radius	CLEVER (average)	Interval	RE(%)
IRIS	100.00	0.001	0.970076	[0.971767, 0.971893]	0.17
	100.00	0.002	1.073300	[1.078274, 1.078550]	0.46
Balance-scale	94.40	0.001	5.378668	[5.387178, 5.390462]	0.16
	96.00	0.002	7.931536	[7.947392, 7.969138]	0.20

7.3 Non-Differentiable Activation Function

We have trained networks with the non-differentiable *ReLU* activation function for IRIS and MNIST. For IRIS, the network has 10 hidden neurons, while for MNIST, we have used 30 neurons in the only hidden layer of the network. The regions to be searched are again of the centre-radius form. In Table 7.2, only the radii are shown.

It is interesting to see that, in this case, acCLEVER indeed returns certified results, and the relative errors are very minute. So, overall, on these cases, CLEVER performs very well.

Table 7.2: Comparison of Lipschitz constant measured by CLEVER and Interval methods with non-differentiable activation function - *ReLU*.

Dataset	Accuracy(%)	Radius	CLEVER (average)	Interval	RE(%)
IRIS	100.00	0.001	22.063391	[22.063389, 22.063389]	9.07e-6
	100.00	0.002	12.200078	[12.200077, 12.200077]	8.20e-6
MNIST	95.29%	0.001	244.398323	[244.398300, 244.398300]	9.41e-6
	95.29%	0.002	244.398323	[244.398300, 244.398300]	9.41e-6

7.4 Critical Regions Near Boundaries

For this experiment, we took the *ReLU* network trained for the IRIS dataset. As there are 4 input neurons and 10 hidden neurons, there are some points in the input space $\mathbb{R}^4$ for which, the values in the hidden layer, just before entering the activation function *ReLU*, are (very close to) zero, for at least 4 neurons in the hidden layer. Taking floating-point errors into account, such values pose quite a challenge to algorithms such as CLEVER.

We have calculated two such points, and the results of the experiments on these points are presented in Table 7.3. The first point has coordinates $(-0.92, -0.90, -0.22, -0.13)$ while the second one has coordinates $(-1.04, -0.71, -1.37, 1.11)$. It should be pointed out that,

while these points may not be valid input values for the particular IRIS task, both CLEVER and our interval algorithms are agnostic of this fact.

As the points lie on the boundaries of critical regions, CLEVER behaves very erratically, and the relative errors are significant. This experiment clearly demonstrates the power of interval methods, and the potential flaws of statistical approaches.

Table 7.3: Comparison of Lipschitz constant measured by CLEVER and Interval methods when the input is near boundaries.

NN Structure	Accuracy (%)	Radius	CLEVER (average)	Interval	RE(%)
4*10*3	100.00	0.001	9.663273	[16.051229, 16.180613]	39.80
	100.00	0.002	6.653347	[9.854075, 10.011593]	32.48

A similar result, with very high relative error was also obtained for the MNIST dataset and its *ReLU* network implementation.

7.5 Finding the Critical Regions

As already pointed out, another benefit of interval computation is that we can locate regions of least robustness, i.e., those with the highest Lipschitz constant. This cannot be done using methods such as CLEVER.

Neural network methods have been used successfully in solving PDE and have numerous advantages compared to the conventional numerical methods such as finite element method [30]. One approach is to sample points from the domain of PDE under consideration and train the neural network on those points. Usually, we select equidistant points in the domain or randomly sample the points as we do not have any information about the solution of the PDE to motivate our choice. It is possible, however, that certain areas of the domain have more complicated topography and sampling more points in those regions will help the neural network better approximate the solution.

Using robustness analysis, we can determine the regions of the domain that lack robustness. We can then adopt a dynamic approach of training the neural networks for PDE solving, as follows:

1. Start with random/equidistant training points in the domain;
2. Train the neural network using these points;
3. Locate the regions of the domain where the neural network lacks robustness;
4. Sample new points in those regions and add them to training points;
5. Continue training the neural network;
6. Repeat steps 3 to 5 until a satisfactory result is obtained.

For the current experiment, we use the penalty method of [31] to solve the following boundary value problem (BVP) on the open square $\Omega := (0,1) \times (0,1)$:

$$\begin{cases} \Delta u(x,y) = 2x(x-1) + 2y(y-1), & (x,y) \in \Omega, \\ u(x,y) = 0, & (x,y) \in \partial\Omega. \end{cases} \quad (7.3)$$

The analytic solution of this problem is given by:

$$u_a(x,y) = x(1-x)y(1-y). \quad (7.4)$$

We trained a neural network with one hidden layer containing 20 neurons on points obtained by discretizing the domain. We sampled 3364 equidistant points inside the domain and 1196 points on the boundary.

Figure 7.1 plots the solution of the PDE (7.3) and Figure 7.2 shows the $\|\cdot\|_1$ -norm of gradient of the solution.

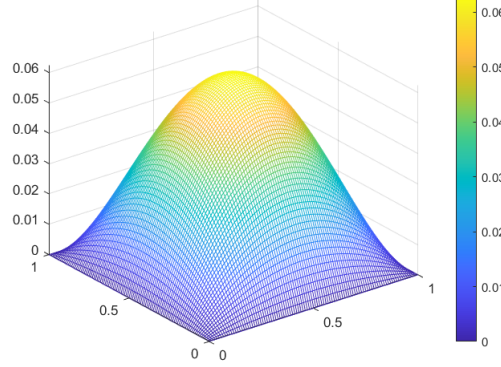


Figure 7.1: The plot for the solution (7.4) of PDE (7.3).

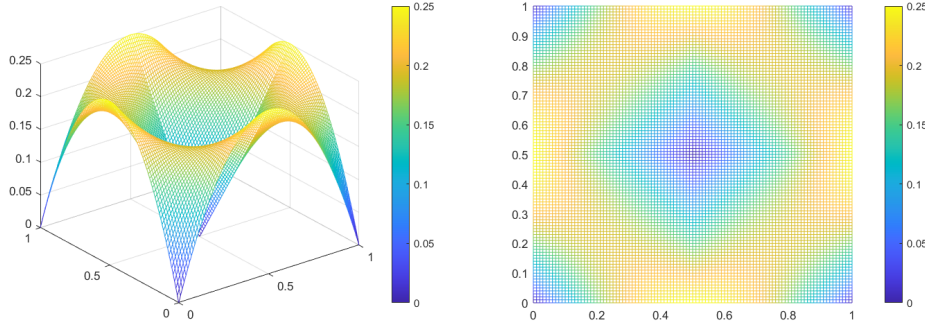


Figure 7.2: The $\|\cdot\|_1$ -norm of the gradient of the solution (7.4) of PDE (7.3).

We consider six different experiments to verify the feasibility of our method. Table 7.4 shows the basic information about experiments. The result presented in Figure 7.3 clearly demonstrates in which parts of the input domain we can obtain the highest Lipschitz constant, which are shown in red color. The result match those shown in Figure 7.2.

Table 7.4: Comparison of Lipschitz constant measured by CLEVER and Interval methods for PDE neural network.

Index	Center Point	Radius	CLEVER (average)	Interval	RE(%)
1	(0.5, 0.5)	0.5	0.229029	[0.253688, 9.378324]	9.72
2	(0.5, 0.5)	0.1	0.072804	[0.095973, 1.778676]	24.14
3	(0.3, 0.3)	0.1	0.188375	[0.190871, 2.038219]	1.31
4	(0.7, 0.7)	0.1	0.189347	[0.192202, 1.928264]	1.49
5	(0.1, 0.1)	0.1	0.186084	[0.191660, 2.114415]	2.91
6	(0.9, 0.9)	0.1	0.181617	[0.192951, 1.869366]	5.87

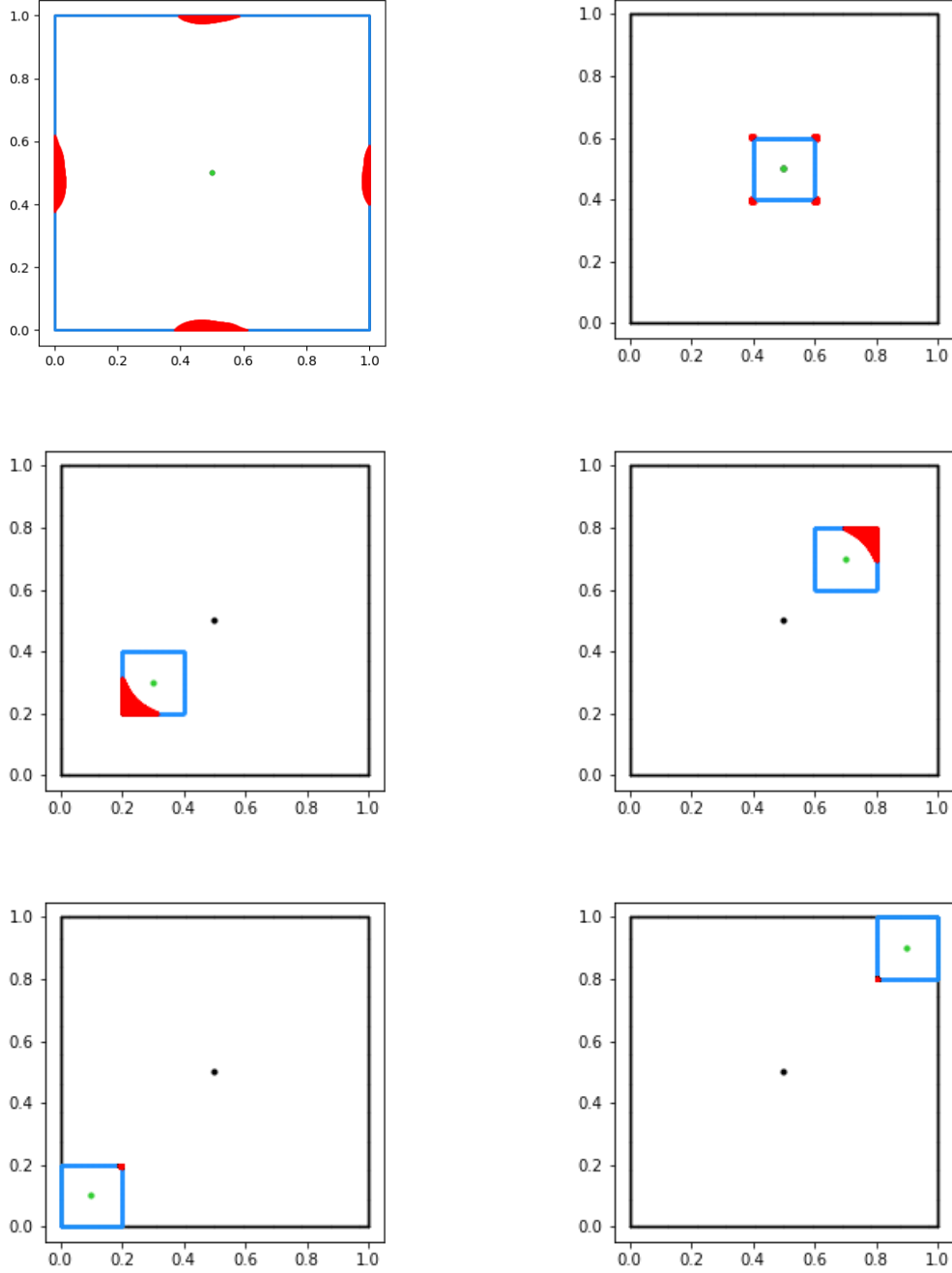


Figure 7.3: Our interval method can locate the regions with the largest Lipschitz constant. The black square is the boundary of the entire domain $(0,1) \times (0,1)$ of the PDE (7.3); the blue square shows the boundary of the region where the search is done and the red parts are the regions where we gain the largest Lipschitz constant. The experiment results match those in Figure 7.2.

7.6 Interval Newton Method

The current aim of experiments of interval Newton Method is to investigate whether this method can be used to find the root(s) of a system of Lipschitz continuous equations as we have discussed in Section 4.1.2. We consider two different experiments for different dimensions. The first one is a one-dimensional function $f(x) = \text{abs}(\cos(x))$ in the range of $[-10, 2]$ which is illustrated in Figure 7.4. This function is Lipschitz continuous everywhere but not differentiable when $x = \pi n - \frac{\pi}{2}, n \in \mathbb{Z}$. However, those non-differentiable points are the roots of the equation as well which cannot be handled by the classical interval Newton

method. Fortunately, our Clarke gradient based Algorithm 3 can provide the solution. When the input range is $[-10, 2]$, this algorithm converges after only six iterations and successfully finds four roots:

$interval([-7.853981633974484, -7.853981633974483])$,
 $interval([-4.712388980384691, -4.71238898038469])$,
 $interval([-1.5707963267948968, -1.5707963267948966])$,
 $interval([1.5707963267948966, 1.5707963267948968])$.

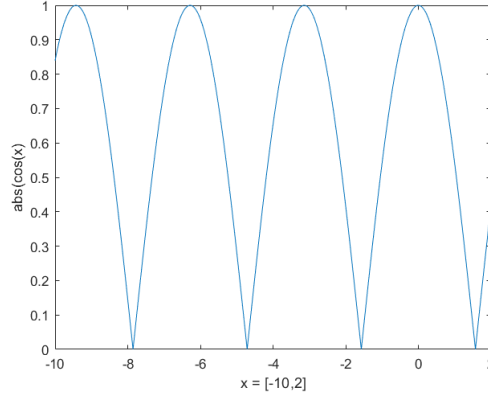


Figure 7.4: Plot of $f(x) = \text{abs}(\cos(x))$ in range of $[-10, 2]$.

In the cases of multi-dimensional inputs, we choose a system of two equations 7.2 with the root $(0, 0)$. Since this system contains absolute function which is Lipschitz continuous but not differentiable at zero, the root of system cannot be dealt by classical Krawczyk operator. Thanks to Clarke gradient, we show that our Algorithm 4 can dispose of this kind of situations. The result $interval([-9.53674e - 07, 9.53674e - 07], [-9.53674e - 07, 9.53674e - 07])$ is calculated after 20 iterations. It should be pointed out, due to the finite representations of interval arithmetic which is discussed in Section 2.3.2, the solution can only get closer to zero with increasing the number of iterations but cannot equal to zero.

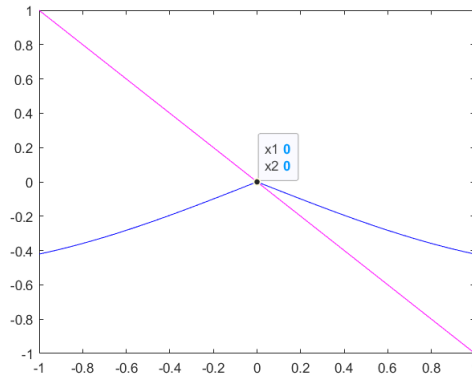


Figure 7.5: Plot of a system of two equations $f(x)$ in range of $[-1, 1]$.

Chapter 8

Summary and Reflections

8.1 Contributions

This project firstly derived a lower bound β_L on the minimum distortion needed to capture an adversarial example from an original example. We indicate that the lower bound β_L is associated with the norm of the local gradients of the original example. As a result, evaluating robustness is reduced to a local Lipschitz constant estimation problem.

Thereby, we proposed a novel interval framework for robustness analysis of neural networks which is based on Lipschitz constant estimation and is truly certified on any given Lipschitz network. The correctness of the method is guaranteed thanks to the chain rule for Clarke gradient (Prop. 2 on page 6), a generalized concept of gradient that is applicable to any Lipschitz map. We present two possible approaches in our framework: multi-dimensional bisection and interval Newton method. Each method has its own set of properties and applicable situations that are covered in Section 4.1.3.

We took multi-dimensional bisection as our main methodology and considered both smooth and non-smooth activation functions. We demonstrated how a straightforward implementation of the method using arbitrary precision interval arithmetic can serve as a verifier for other robustness analysers. Because of the low rate of convergence of the bisection method, if the search space is large, we may only analyse low dimensional cases, such as IRIS and Balance-scale networks of Table 7.1. On the other hand, when the search space is small enough, we can analyse large networks such as MNIST and obtain more accurate results compared with CLEVER, as in Table 7.2. Moreover, due to the nature of interval computation, our method can locate regions of least robustness which has been demonstrated in Section 7.5 and could benefit to robustness enhancement.

Interval Newton method was regarded as a bonus methodology in our framework, in the sense that the original goals of the project proposal had been achieved earlier than planned, thus allowing further exploration of the subject. The interval Newton method has a quadratic rate of convergence, which increases the probability of this method being applied in large networks, even when the search space is large. In contrast to bisection, however, the interval Newton approach is not guaranteed to converge. Furthermore, in the available literature, the method is presented only for the case where the input function is continuously differentiable. Thanks to Clarke gradient again, we can relax this precondition to Lipschitz continuous functions. Section 7.6 demonstrated how interval Newton method successfully computes the roots of a system which is Lipschitz continuous but not differentiable everywhere. It should be pointed out that due to the restricted time, this bonus methodology is only applied on Stage 2 which is discussed in Section 4.1.2. Combining this method with neural network (Stage 1) is one of our future directions.

To emphasize again, estimating the exact Lipschitz constant is a known NP-hard problem [12]. Hence, a method can either be scalable and not certified (e.g., statistical methods), or certified but not scalable (like ours), although, it may be made more efficient on certain types of larger problems. For the methods of the second type, in addition to the ability to obtain ground truths of robustness, more importantly, they can act as verifiers for other methods, like CLEVER. Our framework can be used to identify the robustness

metrics which are prone to significant errors even on smaller networks.

In summary, three general contributions of this project may be highlighted as follows:

1. We proposed a novel interval framework for *certifying* robustness and the correctness of our method is proven by using the chain rule for Clarke gradient. To the best of our knowledge, our method is the first to combine interval computation with Clarke gradient to estimate the *validated* robustness of neural networks.
2. Since our method adopts Clarke’s generalized gradient, our framework is able to analyse neural networks with *non-differentiable* activation functions, e.g., *ReLU*. Besides, because of the property of interval arithmetic, our method can *locate* regions of least robustness which can be used for enhancing robustness.
3. By adopting Clarke gradient, we relax the precondition of interval Newton method—the bonus methodology in our framework—to Lipschitz continuous. To the best of our knowledge, this is the first time that interval Newton method has been successfully used for the case of non-differentiable but Lipschitz continuous functions.

8.2 Future Directions

There are three possible future directions for this project:

1. There is scope for making the bisection algorithm more efficient and applicable to larger networks. To start with, parallelizing the bisection algorithm is straight forward. In a related work [32] which has used interval arithmetic for verifying safety properties of neural networks, a significant speed up has been achieved through the use of symbolic intervals, influence analysis, and monotonicity analysis. Our algorithm can also be made more efficient using these methods.
2. Even with the above enhancements, the core method remains bisection, which is inherently inefficient. Thus, we aim to break away from bisection altogether and adopt our bonus methodology, i.e., interval Newton method. Although we have demonstrated that this method may be applied on non-smooth functions (Stage 2), we have not presented the entire process relating the interval Newton method to robustness analysis of neural networks (Stage 1). In our next step, combining this method with neural network and determining the necessary conditions of convergence are the key points we will focus on.
3. Although Clarke’s generalized gradient is defined for function over any Banach space, we have focused only on input spaces of the form ℓ_∞^n in this project, due to the simpler implementation. Extending the implementation to other ℓ_p^n spaces is also part of our future work.

8.3 Project Management

This project started in the summer of 2020 and is planned on a weekly basis. The project is structured using an Agile-Waterfall hybrid approach. Waterfall is used to complete planning, design, and requirement specification. However, since this project is research-based, the majority of the following tasks are dependent on the outcomes of the preceding tasks. Therefore, each task will be developed and tested in short sprints by using Agile.

The Gantt chart below (Figure 8.1) illustrates the original project plan while Figure 8.2 demonstrates the actual timeline. Compared with the original plan, there is a significant increase of tasks in actual timelines, mainly because the original goals of the project have been achieved earlier than expected, thus allowing for further exploration of this project.

	2020															2021												
	1*	2*	3*	4	5	6	7	8	9	10	11	12	13	14	15*	16*	17	18	19	20	21	22	23	24	25	26	27	28
	Jul	Aug	Sep	10/12	10/19	10/26	11/02	11/09	11/16	11/23	11/30	12/07	12/14	12/21	Exam	Jan	02/01	02/08	02/15	02/22	03/01	03/08	03/15	03/22	03/29	04/05	04/12	04/19
Summer Vacation																												
Learn background knowledge																												
Literature review																												
Explore possible directions																												
Autumn Term																												
Write proposal																												
Learn background knowledge																												
Reproduce related works																												
Supplementary literature review																												
Design bisection method																												
Implement proposed method																												
Get results on smooth function																												
Write interim report																												
Winter Vacation																												
Spring Term																												
Analyse bisection method																												
Learn background knowledge																												
Get results on non-smooth function																												
Locate regions of least robustness																												
Write dissertation																												

Figure 8.1: Planned Timeline

	2020															2021												
	1*	2*	3*	4	5	6	7	8	9	10	11	12	13	14	15*	16*	17	18	19	20	21	22	23	24	25	26	27	28
	Jul	Aug	Sep	10/12	10/19	10/26	11/02	11/09	11/16	11/23	11/30	12/07	12/14	12/21	Exam	Jan	02/01	02/08	02/15	02/22	03/01	03/08	03/15	03/22	03/29	04/05	04/12	04/19
Summer Vacation																												
Learn background knowledge																												
Literature review																												
Explore possible directions																												
Autumn Term																												
Write proposal																												
Learn background knowledge																												
Reproduce related works																												
Supplementary literature review																												
Design bisection method																												
Implement proposed method																												
Get results on smooth function																												
Write interim report																												
Winter Vacation																												
Learn background knowledge																												
Get results on non-smooth function																												
Locate regions of least robustness																												
Supplementary literature review																												
Spring Term																												
Analyse bisection method																												
Learn interval Newton method																												
Design Newton-based algorithm																												
Implement Newton algorithm																												
Make experiments and get results																												
Analyse Newton method																												
Write dissertation																												

Figure 8.2: Actual Timeline

Most of the literature review and mathematics background knowledge learning were completed during summer vacation. In the autumn semester, all planned tasks were completed smoothly and on time. Then, I took advantage of the winter vacation to supplement the experiments needed for this project. Thereby, I achieved all original goals earlier than planned and have time to investigate another possible approach to make the proposed framework more efficient in the spring semester.

8.4 Reflections

Overall, the speed of the project development is satisfactory and deadlines are strictly maintained by all deliverable submitted on time. All the tasks planned for this project have been achieved and additional aims and objectives have been accomplished.

Specifically, in the summer vacation and first semester, the majority of time was spent on studying technique background knowledge and researching related literature in detail to explore the possible directions. Fortunately, we determined to take the impact of Lipschitz constant on robustness as a breakthrough. Then, we successfully designed a novel interval framework with a straightforward implementation to serve as a verifier for other cutting-edge robustness analysers and demonstrated some of those metrics are easily prone to errors.

However, there still have some room for improvement in the first semester. Firstly, since the code of this project is written in C++ and python and various libraries are introduced for specific purposes, these cause high complexity of the program. As a result, codes for this project sometimes have several hard-to-find errors. In the first semester, significant time was spent dealing with unexpected bugs in codes. In this regard, I introduced unit testing to ensure the accuracy of each function. Due to the high demand for related technical knowledge, I discovered that I lacked sufficient related knowledge, which rendered some tasks cannot be finished on time. Therefore, more time should be allocated to reviewing the literature and learning background knowledge. Moreover, leaving only one week to write the interim report is inadequate, and it caused me to struggle during that week. A more reasonable time allocation was expected.

I took those lessons and put forward as well as applied the mitigated methods to decrease the risk in the second semester. The unit test guarantees the correctness of my program and saved me a significant amount of time to solve unexpected bugs. Moreover, I took advantage of the winter vacation to read and learn more relevant background knowledge. More supplementary experiments on the bisection method were carried out in the vacation as well. Furthermore, I maintained the extra time as the buffering for the difficult tasks and allotted plenty of time to complete the dissertation in the second semester. All these enhancements made my project in the second semester run more smooth and leisurely.

To summarize, the following table demonstrates the discovered risks in the first semester and how those risks were mitigated in the second semester. Those risks will be kept in mind and will be avoided in future planning. This project has been a meaningful and memorable experience for me that I have broadened my knowledge in related fields and strengthened my skills for critical thinking, planning, coding as well as technical writing.

Table 8.1: Risks and Mitigated Methods

Risks in the autumn semester	Mitigated methods in the spring semester
Codes have several hard-to-find errors	Take unit testing
Lack adequate background knowledge	Review more related field research literature.
	Discuss the puzzle with supervisor.
Some difficult tasks take more time than I planned	Maintain the extra time for buffering.
	Be more cautious when designing plan.
The time left for writing the interim report is too short	Start writing the final report early

References

- [1] D. Heaven, “Why deep-learning ais are so easy to fool,” *Nature*, vol. 574, no. 7777, pp. 163–166, 2019.
- [2] T.-H. Chan, K. Jia, S. Gao, J. Lu, Z. Zeng, and Y. Ma, “Pcanet: A simple deep learning baseline for image classification?” *IEEE transactions on image processing*, vol. 24, no. 12, pp. 5017–5032, 2015.
- [3] Z.-Q. Zhao, P. Zheng, S.-t. Xu, and X. Wu, “Object detection with deep learning: A review,” *IEEE transactions on neural networks and learning systems*, vol. 30, no. 11, pp. 3212–3232, 2019.
- [4] A. Mathis, P. Mamidanna, K. M. Cury, T. Abe, V. N. Murthy, M. W. Mathis, and M. Bethge, “Deeplabcut: markerless pose estimation of user-defined body parts with deep learning,” *Nature neuroscience*, vol. 21, no. 9, pp. 1281–1289, 2018.
- [5] A. Nguyen, J. Yosinski, and J. Clune, “Deep neural networks are easily fooled: High confidence predictions for unrecognizable images,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 427–436.
- [6] S. Ren, K. He, R. Girshick, and J. Sun, “Faster r-cnn: Towards real-time object detection with region proposal networks,” in *Advances in neural information processing systems*, 2015, pp. 91–99.
- [7] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2016, pp. 770–778.
- [8] F. Schroff, D. Kalenichenko, and J. Philbin, “Facenet: A unified embedding for face recognition and clustering,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 815–823.
- [9] J. Su, D. V. Vargas, and K. Sakurai, “One pixel attack for fooling deep neural networks,” *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 5, pp. 828–841, 2019.
- [10] T.-W. Weng, H. Zhang, P.-Y. Chen, J. Yi, D. Su, Y. Gao, C.-J. Hsieh, and L. Daniel, “Evaluating the robustness of neural networks: An extreme value theory approach,” *arXiv preprint arXiv:1801.10578*, 2018.
- [11] M. Jordan and A. G. Dimakis, “Exactly computing the local lipschitz constant of relu networks,” *arXiv preprint arXiv:2003.01219*, 2020.
- [12] K. Scaman and A. Virmaux, “Lipschitz regularity of deep neural networks: analysis and efficient estimation,” *arXiv preprint arXiv:1805.10965*, 2018.
- [13] R. Paulavičius and J. Žilinskas, “Analysis of different norms and corresponding lipschitz constants for global optimization in multidimensional case,” *Information Technology and Control*, vol. 36, no. 4, 2007.
- [14] F. H. Clarke, *Optimization and nonsmooth analysis*. SIAM, 1990.

- [15] S. Kakade and J. D. Lee, “Provably correct automatic subdifferentiation for qualified programs,” *arXiv preprint arXiv:1809.08530*, 2018.
- [16] R. E. Moore, R. B. Kearfott, and M. J. Cloud, *Introduction to interval analysis*. SIAM, 2009.
- [17] N. Revol and F. Rouillier, “Mpmc: a library for arbitrary precision interval arithmetic,” 2002.
- [18] I. Goodfellow, “Gradient masking causes clever to overestimate adversarial perturbation size,” *arXiv preprint arXiv:1804.07870*, 2018.
- [19] M. Hein and M. Andriushchenko, “Formal guarantees on the robustness of a classifier against adversarial manipulation,” in *Advances in Neural Information Processing Systems*, 2017, pp. 2266–2276.
- [20] M.-I. Nicolae, M. Sinn, M. N. Tran, B. Buesser, A. Rawat, M. Wistuba, V. Zantedeschi, N. Baracaldo, B. Chen, H. Ludwig, I. Molloy, and B. Edwards, “Adversarial robustness toolbox v1.2.0,” *CoRR*, vol. 1807.01069, 2018. [Online]. Available: <https://arxiv.org/pdf/1807.01069>
- [21] M. L. Bianchi, S. V. Stoyanov, G. L. Tassinari, F. J. Fabozzi, S. M. Focardi *et al.*, “Extreme value theory,” *World Scientific Book Chapters*, pp. 367–430, 2019.
- [22] L. Weng, H. Zhang, H. Chen, Z. Song, C.-J. Hsieh, L. Daniel, D. Boning, and I. Dhillon, “Towards fast computation of certified robustness for relu networks,” in *International Conference on Machine Learning*. PMLR, 2018, pp. 5276–5285.
- [23] M. Fazlyab, A. Robey, H. Hassani, M. Morari, and G. J. Pappas, “Efficient and accurate estimation of lipschitz constants for deep neural networks,” *arXiv preprint arXiv:1906.04893*, 2019.
- [24] N. Kamath, I. Voiculescu, and C. K. Yap, “Empirical study of an evaluation-based subdivision algorithm for complex root isolation,” in *Proceedings of the 2011 International Workshop on Symbolic-Numeric Computation*, 2012, pp. 155–164.
- [25] A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala, “Pytorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, H. Wallach, H. Larochelle, A. Beygelzimer, F. d'Alché-Buc, E. Fox, and R. Garnett, Eds. Curran Associates, Inc., 2019, pp. 8024–8035. [Online]. Available: <http://papers.neurips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.pdf>
- [26] A. G. Baydin, B. A. Pearlmutter, A. A. Radul, and J. M. Siskind, “Automatic differentiation in machine learning: a survey,” *Journal of machine learning research*, vol. 18, 2018.
- [27] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE transactions on neural networks*, vol. 9, no. 5, pp. 987–1000, 1998.
- [28] B. Schäling, *The boost C++ libraries*. Boris Schäling, 2011.

- [29] W. Jakob, J. Rhinelanders, and D. Moldovan, “pybind11 seamless operability between c++11 and python,” 2016, <https://github.com/pybind/pybind11>.
- [30] I. E. Lagaris, A. Likas, and D. I. Fotiadis, “Artificial neural networks for solving ordinary and partial differential equations,” *IEEE Transactions on Neural Networks*, vol. 9, no. 5, pp. 987–1000, 9 1998.
- [31] I. E. Lagaris, A. C. Likas, and D. G. Papageorgiou, “Neural-network methods for boundary value problems with irregular boundaries,” *IEEE Transactions on Neural Networks*, vol. 11, no. 5, pp. 1041–1049, 9 2000.
- [32] S. Wang, K. Pei, J. Whitehouse, J. Yang, and S. Jana, “Formal security analysis of neural networks using symbolic intervals,” in *Proceedings of the 27th USENIX Conference on Security Symposium*, ser. SEC’18. USENIX Association, 2018, pp. 1599–1614.